HAPTION
Atelier Relais
ZA – Route de Laval
53210 SOULGE SUR OUETTE
FRANCE
http://www.haption.com
contact@haption.com

VIRTUOSE API V4.04

DOCUMENTATION

TABLE OF CONTENTS

1 INTRODUCTION

1.1 What is the VIRTUOSE API?

The VIRTUOSE API (Application Programming Interface) is a library of functions written in the C programming language, allowing programmers to interface their applications with haptic devices of the VIRTUOSE series.

The VIRTUOSE API supports the following devices:

- VIRTUOSE 6D
- VIRTUOSE 3D
- VIRTUOSE 6D DESKTOP
- VIRTUOSE 3D DESKTOP

It is able to control several devices at the same time, as well as the simulation software itself.

It is available for different hardware architectures and operating systems:

- PC-type computer running Microsoft Windows
- PC-type computer running Linux

1.2 Composition of this documentation

This document includes the following chapters:

1. Introduction (this chapter)
2. Installation manual
3. User manual
4. Reference manual
5. Glossary

2 INSTALLATION MANUAL

The VIRTUOSE API is a library of C functions. Its installation consists simply to including files into the development environment. Those header and binary files depend on the operating system:

- For Microsoft Windows:
  - virtuoseAPI.h
  - virtuoseDLL.lib
  - virtuoseAPI.dll
- For Linux:
  - virtuoseAPI.h
  - virtuoseAPI.so
  - libvirtuose.a

3 USER MANUAL

3.1 Contents of the Virtuose API

The VIRTUOSE API is composed of the following files:

- One header file for C/C++ to be included in your source code: VirtuoseAPI.h
- One static library: VirtuoseDLL.lib (Windows) or libvirtuose.a (Linux)
- One dynamic library: VirtuoseAPI.dll (Windows) or virtuoseAPI.so (Linux)

3.2 Conventions

3.2.1 Programming

The functions of VIRTUOSE API respect the following rules:

- The names of functions are in English language; they start with the letters *"virt"* and don't include any underscore, but all following words are capitalized (ex: *"virtGetPosition"*).
- Apart from the two functions *"virtOpen"* and *"virtGetErrorMessage"*, all functions need as first parameter an object of type "*VirtContext*", which is created by calling *"virtOpen"* and destroyed with *"virtClose"*.
- Apart from *"virtOpen"*, *"virtGetErrorCode"* and *"virtGetErrorMessage"*, all functions return 0 in case of success and -1 otherwise.
- The functions use C prototypes; the header file VirtuoseAPI.h includes an *extern "C"* pragma for use with C++ applications.

3.2.2 Interaction mode

The VIRTUOSE library implements different control modes for the haptic device. The choice of the control mode depends on the application:

1. Force/position control: the application sends forces and torques to the device and reads the position and speed of the end-effector frame.
2. Position/force control: this advanced control mode allows direct coupling with virtual objects; in that case, the application sends the position and speed of the centre of the object to the device, and reads the forces and torques to be applied to the object for dynamic integration. Stiffness and damping are calculated by the embedded software, knowing the mass and inertia of the object, in order to ensure control stability.
3. Position/force with virtual guides: this is the same as above, with addition of virtual guides (e.g. fixed translation, fixed rotation, etc.).

3.2.3 Definition of reference frames

All geometric entities (position, speed, forces and torques, etc.) are defined with respect to a reference frame.

By definition, all reference frames used by the VIRTUOSE API are right-handed. This is different to the conventions used in several graphic libraries. Chapter 4 deals with the change of conventions. By default, all reference frames are defined as follows:

- The origin is the intersection point of joint 1 (base rotation) and joint 2 (first vertical movement) of the haptic device.
- The X axis is horizontal, aligned with the central position of joint 1 and of the haptic device, pointing towards the user.
- The Y axis is horizontal, orthogonal to the X axis and pointing to the right of the user.
- The Z axis is vertical, pointing upwards.

The library defines the following reference frames:

1. The environment frame corresponds to the origin of the virtual scene; it is specified by the software application independently of the Virtuose API.
2. The observation frame corresponds generally to the position of the camera, and is defined with respect to environment frame.
3. The base frame represents the centre of the haptic device, and is defined with respect to the observation frame.
4. The tool frame corresponds to the “wrist” of the haptic device, i.e. the intersection of joints 5 and 6, and is defined with respect to the environment frame.

The transformation of the following frame should be defined only once using the API, before starting the periodic update loop:

- Base frame (with respect to the observation frame)

Setting the transformation of the observation frame will immediately affect the current transformation of the tool frame.

The position of the following frame can be modified dynamically using the API:

- Observation frame (with respect to the environment frame)

Setting the transformation of the observation frame will not immediately affect the current transformation of the tool frame, but its evolution afterwards.

The transformation of the following frame cannot be modified:

- Tool frame (with respect to the environment frame)

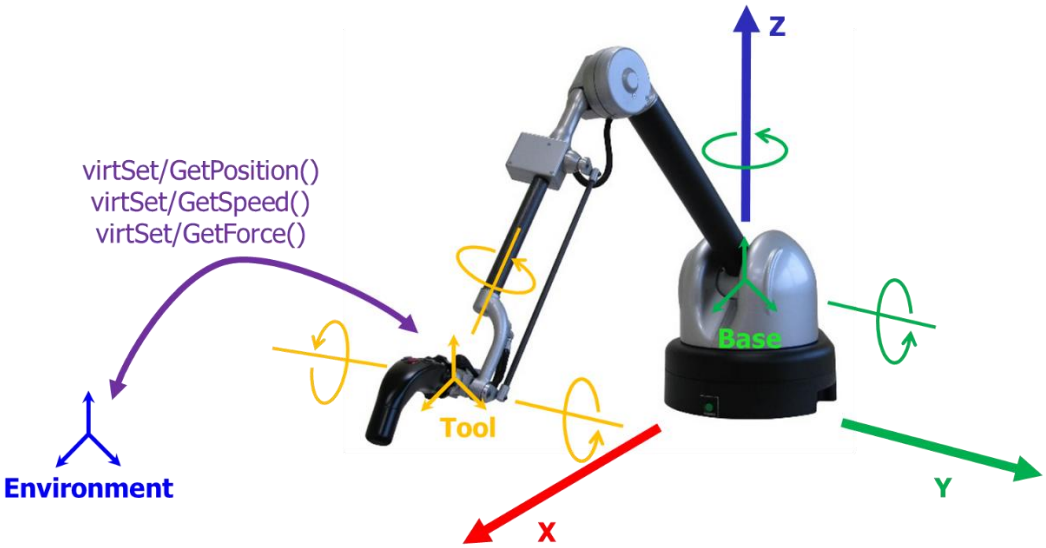


Figure 1 - Virtuose base frame

3.2.4 Physical units

All values used in the API are expressed in physical units using metric conventions:

- Durations in seconds (s)
- Dimensions in meters (m)
- Angles in radians (rad)
- Linear velocities in meters per second ($m.s^{-1}$)
- Angular velocities in radians per second ($rad.s^{-1}$)
- Forces in Newtons (N)
- Torques in Newton-meters (N.m)
- Masses in kilogrammes (kg)
- Inertia components in $kg.m^2$

3.2.5 Expression of geometric entities

The API uses different geometric entities:

- Positions are expressed as displacement vectors with seven components, i.e. one translation term (x, y, z) followed by one rotation term in the form of a normalized quaternion (qx, qy, qz, qw) (cf. glossary).
- Velocities are expressed as cinematic tensors (vx, vy, vz, wx, wy, wz) (cf. glossary)
- Efforts are expressed as dynamic tensors (fx, fy, fz, cx, cy, cz) (cf. glossary)

3.3 Implementation of the VIRTUOSE API

3.3.1 Initialisation

The first step in use of VIRTUOSE API is to establish the communication with the haptic device. The `virtOpen` function carries out that operation, allocates the memory for storing internal data and returns a pointer of type `VirtContext`. This pointer must be used for calling all other functions: In case several haptic devices are connected to the application, the `VirtContext` object also identifies which VIRTUOSE to address a request to.

It is essential to test the return value of the `virtOpen` function, the API works well only if the opening of the communication is successful:

```
VirtContext VC;
VC = virtOpen("127.0.0.1#53210");
if (VC == NULL)
{
fprintf(stderr, "Erreur dans virtOpen: %s\n",
     virtGetErrorMessage(virtGetErrorCode(NULL)));
return -1;
}
```

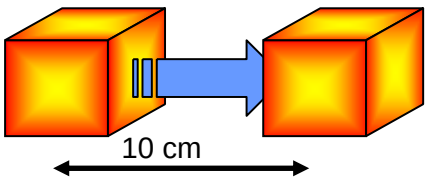
Afterwards, we advise to do explicit configuration of the control mode, as the default parameters possibly don't correspond to your application needs:

1. The speed factor (`virtSetSpeedFactor` function)

   The role of speed factor is to modify the amplitude of translations during the simulation. This function is useful when the virtual scene is much larger or smaller as the workspace of the Virtuose haptic device.

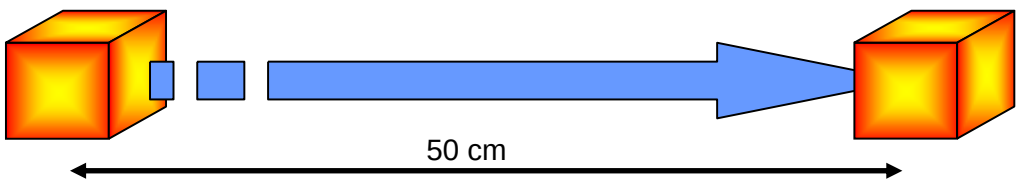
   The following figure illustrates the effect of the speed factor on a virtual object:

   Case 1: Speed factor of 1.0

   For a 10 cm horizontal displacement of haptic interface, the displacement of the virtual box is the same.

   Case 2: Speed factor of 5.0

For a 10 cm horizontal displacement of haptic interface, the displacement of the virtual box is 5 times more, i.e. 50 cm.

2. The force factor (`virtSetForceFactor` function)

   The role of the force factor is to modify the level of forces to be exerted by the user when moving virtual objects, especially in the case of large or heavy objects. A force factor larger than 1 will amplify the forces computed by the simulation, while a value lower than 1 will amplify the forces exerted by the user on the virtual environment.

3. The indexing mode (`virtSetIndexingMode` function)

   When the user reaches the limits of the device workspace, he can use the indexing function (also called offset) in order to come back to a more appropriate position without moving the virtual object he is attached to. The indexing function is also activated when the user lets go of the proximity sensor (also called dead-man sensor), or when the power is switched off.

   Three different modes of indexing are defined:

   - `INDEXING_ALL` authorizes indexing on all movements, i.e. rotations and translations.
   - `INDEXING_ALL_FORCE_FEEDBACK_INHIBITION` authorizes indexing on all movements, and removes the force-feedback while indexing is active.
   - `INDEXING_TRANS` authorizes indexing only on translations, i.e. the orientation of the object is always identical to that of the device end-effector
   - `INDEXING_NONE` forbids indexing on all movements.

4. The simulator integration timestep (`virtSetTimeStep` function)
5. The position of the base reference frame with respect to the observation reference frame (`virtSetBaseFrame` function)
6. The position of the observation reference frame with respect to the environment reference frame (`virtSetObservationFrame` function)
7. The type of control mode (`virtSetCommandType` function)

The following source code represents very common settings:

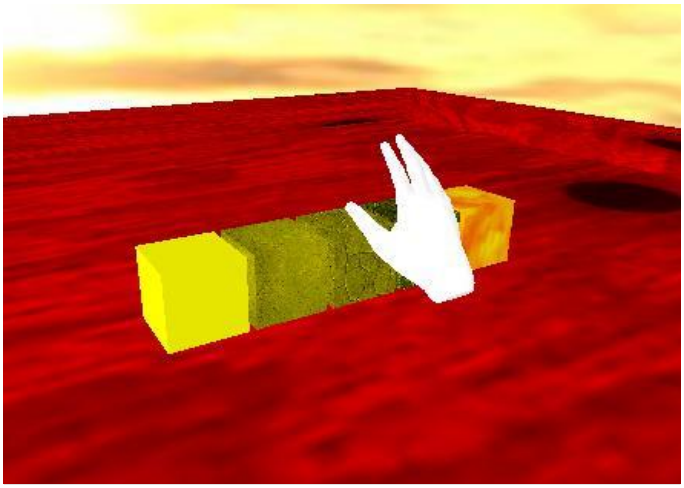
```
float identity[7] = {0.0f,0.0f,0.0f,0.0f,0.0f,0.0f,1.0f};
virtSetIndexingMode(VC, INDEXING_ALL);
virtSetForceFactor(VC, 1.0f);
virtSetSpeedFactor(VC, 1.0f);
virtSetTimeStep(VC, 0.003f);
virtSetBaseFrame(VC, identity);
virtSetObservationFrame(VC, identity)
virtSetCommandType(VC, COMMAND_TYPE_VIRTMECH);
```

Finally, the initialisation phase ends with the enabling of force-feedback (`virtSetPowerOn` function). After that step, the user can activate the force-feedback at any time, by switching the power switch to ON and engaging the proximity sensor:

```
virtSetPowerOn(VC, 1);
```

3.3.2 Use as a pointer

In many applications, it is necessary to "pointer mode", in which the current position of the haptic device is shown on the screen as a graphic object (also called "avatar"). This is useful for navigating in the virtual scene in order to choose an object to grasp, or else to call a function using a contextual menu.



We define a simple update function, which will be called at regular intervals, in order to read the position of the tool frame of the haptic device and send new control values. The function will be called by the simulation, for example as a call back.

In simple `COMMAND_TYPE_IMPEDANCE` control mode, the function must set the control force to 0, and return the position of the tool frame. It can be written as follows:

```
int update_avatar(VirtContext VC, float *position)
{
float null [6] = {0.0f,0.0f,0.0f,0.0f,0.0f,0.0f};
virtSetForce(VC, null);
virtGetPosition(VC, position);
return 0;
}
```

In `COMMAND_TYPE_VIRTMECH` control mode, the function must read the position and speed of the tool frame, and send it back to the device (as a consequence, the position and speed errors are both null, and no force is generated):

```
int update_avatar(VirtContext VC, float *position)
{
float speed[6];
virtGetPosition(VC, position);
virtGetSpeed(VC, speed);
virtSetPosition(VC, position);
virtSetSpeed(VC, speed);
return 0;
}
```

In both cases, the function returns the current position of the avatar in the parameter `position`, in translation and rotation. Its value is affected by the choice of the base and observation reference frames, and by the indexing: when the offset push-button is pressed, the avatar does not follow the movements of the haptic device anymore.

3.3.3 Object attachment

In `COMMAND_TYPE_VIRTMECH` control mode, it is possible to attach the tool frame of the VIRTUOSE directly to a virtual object, and the haptic device is then seen by the simulation as a dynamic bilateral constraint applied on the object.

The attachment to a virtual object is done as follows:

1. The function `virtAttachVO` carries out the attachment, taking into account the mass and inertia of the object as well as the integration timestep of the simulation, in order to compute optimal values for the control parameters, guaranteeing the stability.
2. The position and speed of the reference point of the object are then sent as control values using the functions `virtSetPosition` and `virtSetSpeed`.

The following source code implements the attachment of an object of mass `m`, of inertia `mxmymz`, which centre of mass is at the position `P` moving with speed `V`:

```
virtAttachVO(VC, m, mxmymz);
virtSetPosition(VC, P);
virtSetSpeed(VC, S);
```

In order to enable the motion of the virtual object, we define an update function to be called at a fixed rate by the simulation, typically as a call back:

```
int update_object(VC,float *P, float *S, float *force)
{
virtSetPosition(VC, P);
virtSetSpeed(VC, S);
virtGetForce(VC, force);
return 0;
}
```

After calling the function, the simulation should add the value of `force` to the constraints active on the object, before integrating the laws of physics.

The reference point of the object is normally its center of mass. It is possible to change the point of attachment by using the function `virtSetCatchFrame`.

3.3.4 Timing

Sometimes it is necessary to add software temporizations between writing and reading operations of a parameter, as the embedded software controller of the Virtuose needs time to taking the new value into account. This concern:

- the base frame of a virtual mechanism,
- the sequences of two virtual mechanisms

Example: Cartesian movement from current position of end-effector 20 cm along X axis, then a rotation of X axis from effector.

```
float virtVmBaseFrame[7];
virtVmSetType(VC, VM_TYPE_CartMotion);
// the current position of end-effector becomes
// the base frame of the virtual mechanism
virtVmSetBaseFrameToCurrentFrame(VC);
Sleep(10);
// new modification of base frame
virtGetVmBaseFrame(VC, virtVmBaseFrame);
virtVmBaseFrame[0] += 0.2;
virtSetVmBaseFrame(VC, virtVmBaseFrame);
// activation of guide
virtVmActivate(VC);
// wait bound
virtVmWaitUpperBound(VC);
virtVmDeactivate(VC);
Sleep(10);
// new mechanism, rotation of x axis
virtVmSetType(VC, VM_TYPE_Rx);
virtVmActivate(VC);
```

3.3.5 Evolution post-3.98

Since version 3.98, the header “VirtuoseAPI.h” has changed radically. However, we have made our best effort so that previous application code will recompile without any need to change it.

It is even possible to use newer versions of the VirtuoseAPI with old application code without recompiling it, just by copying the new version of the DLL.

3.4 Problem solving

We describe below some typical problems, which arise when using the API. For each of them, we give solutions and rules to follow.

3.4.1 Instability

The VIRTUOSE API computes optimal values for the control parameters, with respect to two criteria:

1. Global stability of the system (simulation – virtual object – haptic device)
2. Stiffness of the haptic device control loop

Should an instability occur nonetheless, it is advisable to verify several constraints:

- Is the integration step of the simulation exactly identical to that given to the API by calling the function `virtSetTimeStep`?
- Is the simulation on time, i.e. is there no delay between the simulated time and the real time?
- Are the mass and inertia parameters of the virtual object exactly identical to those given to the API?

Moreover, some collision detection algorithms give very noisy results, with ill-defined surface normal, which can be misinterpreted as instability.

3.4.2 Abnormal sticking

This phenomenon is characteristic of a loss of efficiency in the connection with the Virtuose. This is particularly true when the Virtuose is situated on a network branch shared with many other workstations, or when the Virtuose and the simulator are not on the same branch but talk to each other over several Ethernet relays.

In all those cases, it is necessary to modify the network organization, the best solution being to dedicate only network interface to the Virtuose.

3.4.3 Low stiffness

This phenomenon appears when the Virtuose is attached to very large objects, or else when the movement scale factor (`virtSetSpeedFactor`) is inadequate. As a consequence, the movement of the attached object is difficult to control, because of its very high mass and inertia compared to the forces exerted by the VIRTUOSE.

If that is the case, then it is advisable to reduce the mass and inertia of the attached object, either by modifying its size (global scale factor for the whole scene), or by diminishing its density. It is also possible to reduce the force scale factor (`virtSetForceFactor`), which results in amplifying the forces exerted by the VIRTUOSE, (see below).

3.4.4 Low force feedback

This phenomenon appears when the force scale factor is too low (`virtSetForceFactor`). It can also occur with some combinations of mass, inertia and movement scale factor.

In all cases, it is advisable to raise the force factor, or else to modify the object in order to bring it closer to the optimal values (sphere of 10 cm and 300 g, speed factor of 1).

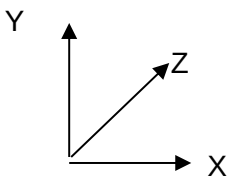
4 CHANGE OF CONVENTIONS

A classical problem is the transformation between conventions used in computer graphics (left-handed coordinate system, Z pointing towards the back of the screen) and those used in robotics (right-handed coordinate system, Z vertical and pointing upwards).

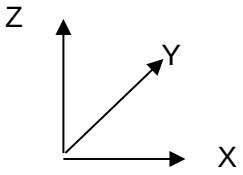
We propose here below a simple conversion scheme for a “graphic engine” (GE).

4.1 Example

We define the following conversion functions, which switch axes Y and Z, in order to transform between computer graphics and robotics conventions to and fro:



Vortex coordinates



Virtuose coordinates

We further suppose that:

- GE positions are defined as positions and quaternions.
- GE speeds are defined as linear speed and angular speed.

```
void transformGEPosQuatToVirtPosEnv(
                                float *gePos,
                                float *geQuat,
                                float *virtPosEnv)
{
      virtPosEnv[0] = gePos[0];
      virtPosEnv[1] = gePos[2];
      virtPosEnv[2] = gePos[1];
      virtPosEnv[3] = -geQuat[1];
      virtPosEnv[4] = -geQuat[3];
      virtPosEnv[5] = -geQuat[2];
      virtPosEnv[6] = geQuat[0];
}

void transformVirtPosEnvToGEPosQuat(
                                float *virtPosEnv,
                                float *gePos,
                                float *geQuat)
{
      gePos[0] = virtPosEnv[0];
      gePos[1] = virtPosEnv[2];
      gePos[2] = virtPosEnv[1];
      geQuat[0] = virtPosEnv[6];
      geQuat[1] = -virtPosEnv[3];
      geQuat[2] = -virtPosEnv[5];
      geQuat[3] = -virtPosEnv[4];
}
void transformGELinAngVelToVirtSpeedEnv(
```

```
                                    float *geLinVel,
                                    float *geAngVel,
                                    float *virtSpeedEnv)
{
      virtSpeedEnv[0] = geLinVel[0];
      virtSpeedEnv[1] = geLinVel[2];
      virtSpeedEnv[2] = geLinVel[1];
      virtSpeedEnv[3] = -geAngVel[0];
      virtSpeedEnv[4] = -geAngVel[2];
      virtSpeedEnv[5] = -geAngVel[1];
}

void transformVirtTorsToGEForceTorque(
                                    float *virtTors,
                                    float *geForce,
                                    float *geTorque)
{
      geForce[0] = virtTors[0];
      geForce[1] = virtTors[2];
      geForce[2] = virtTors[1];
      geTorque[0] = -virtTors[3];
      geTorque[1] = -virtTors[5];
      geTorque[2] = -virtTors[4];
}

void transformGEInertiaTensorToVirtInertiaTensor(
                                    float *geTensor,
                                    float *virtTensor)
{
      virtTensor[0] = geTensor[0];
      virtTensor[1] = geTensor[2];
      virtTensor[2] = geTensor[1];
      virtTensor[3] = geTensor[6];
      virtTensor[4] = geTensor[8];
      virtTensor[5] = geTensor[7];
      virtTensor[6] = geTensor[3];
      virtTensor[7] = geTensor[5];
      virtTensor[8] = geTensor[4];
}
```

5 REFERENCE MANUAL

5.1 Function index by alphabetical order

5.2 Connection management

NAME

virtOpen – Open a connection to the controller of the Virtuose.

SYNOPSIS

```
#include "virtuoseAPI.h"
VirtContext virtOpen(const char *host);
```

DESCRIPTION

The `virtOpen` function opens a connection to the controller of the Virtuose.

PARAMETERS

The `host` parameter corresponds to the URL (*Uniform Resource Locator*) of the VIRTUOSE controller to connect to.
In the current version of the API, only one type of communication protocol is available, therefore the URL is always in the form:

```
"udpxdr://identification:port_number+interface"
```

`udpxdr` is the only protocol available in the current version of the API.
`identification` should be replaced by the host name of the VIRTUOSE controller if it can be resolved by a DNS, or else by its IP address in dotted form (e.g. "192.168.0.1").
`port_number` should be replaced by the port number to be used by the API to connect to the Virtuose controller. The default value is 0, and in that case the API looks for a free port number starting from 3131.
`interface` designates the physical interface to be used by the API (ignored in the case of `udpxdr`).

In case only `identification` is given, the URL is completed as follows:

```
.... "udpxdr://identification:0"
```

Note: the automatic completion is limited to `udpxdr` only.
The initial prefix "url:" defined in the URL standard is supported but ignored.

RETURN VALUE

In case of success, the `virtOpen` function returns a pointer of type `VirtContext` (pointer to a hidden data structure). Otherwise, it returns `NULL` and the `virtGetErrorCode(NULL)` function gives access to an error code.

ERRORS

`VIRT_E_SYNTAX_ERROR`
The `host` parameter does not correspond to a valid URL.

`VIRT_E_COMMUNICATION_FAILURE`
The `virtOpen` function did not succeed in setting up a connection to the Virtuose.

`VIRT_E_OUT_OF_MEMORY`
The `virtOpen` function could not allocate enough memory for the object `VirtContext`.

`VIRT_E_INCOMPATIBLE_VERSION`
The Virtuose controller version is not compatible with the one of the API.

SEE ALSO

virtClose

virtGetErrorCode

NAME

virtClose – Closing of connection to the controller of the VIRTUOSE.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtClose(VirtContext VC);
```

DESCRIPTION

The `virtClose` function closes the connection to the controller of the VIRTUOSE.

PARAMETERS

The `VC` parameter corresponds to the object of type `VirtContext` returned by the `virtOpen` function.

RETURN VALUE

In case of success, the `virtClose` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtOpen

virtGetErrorCode

5.3 Initialization

NAME

virtAPIVersion – Read the Virtuose library version.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtAPIVersion(VirtContext VC, int * major, int * minor);
```

DESCRIPTION

The `virtAPIVersion` function allows reading the version number of the Virtuose library.

PARAMETERS

The `VC` parameter corresponds to the object of type `VirtContext` returned by the `virtOpen` function.
The `major` parameter is a pointer to an integer. It corresponds to the major index of the software version.
The `minor` parameter is a pointer to an integer. It corresponds to the minor index of the software version.

RETURN VALUE

In case of success, the `virtClose` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtOpen

virtGetErrorCode

NAME

virtGetControlerVersion – Read the version of the embedded software.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetControlerVersion(VirtContext VC,
                            int* major, int* minor);
```

DESCRIPTION

The `virtGetControlerVersion` function reads the version of the embedded software of the controller.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `major` parameter corresponds to the major index of the version.
The `minor` parameter corresponds to the minor index of the version.

RETURN VALUE

In case of success, the `virtGetControlerVersion` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtOpen

virtGetErrorCode

NAME

virtSetOutputFile – Define the output file.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetOutputFile(VirtContext VC, char * name);
```

DESCRIPTION

The `virtSetOutputFile` function creates a text file used for sending message during device utilization. If the file already exists, the file will be erased and the content lost.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `name` parameter is a string. It corresponds to the name of the created file.

RETURN VALUE

In case of success, the `virtGetControlerVersion` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The parameter `VC` does not correspond to a valid `VirtContext` object.

SEE ALSO

virtOpen

virtGetErrorCode

NAME

virtOutputsSettings – Set digital outputs value

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtOutputsSetting(VirtContext VC, unsigned int outputs);
```

DESCRIPTION

The `virtOutputsSettings` function sets the Virtuose digital output values.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `outputs` parameter is an integer. It corresponds to the new digital outputs value.

RETURN VALUE

In case of success, the `virtOutputsSetting` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The parameter `VC` does not correspond to a valid `VirtContext` object.

SEE ALSO

NAME

virtSetPwmOutput – Project specific

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetPwmOutput(VirtContext VC, float *pwm);
```

DESCRIPTION

The `virtSetPwmOutput` function was define for the specific. It should not be used for any standard device.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `pwm` parameter is a table of 7 floating point numbers.

RETURN VALUE

In case of success, the `virtSetPwmOutput` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The parameter `VC` does not correspond to a valid `VirtContext` object.

SEE ALSO

NAME

virtDisableControlConnexion – Disable the watchdog communication control between the virtuoseAPI and the controller.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtDisableControlConnexion (VirtContext VC, int disable);
```

DESCRIPTION

The `virtDisableControlConnexion` function disables (or enables) the watchdog control on the communication with the Virtuose controller. The role of the watchdog control is to stop force-feedback in case of a software failure on the simulation side. In practice, if the simulation does not update the control values during a time period of 2 seconds, the Virtuose considers that the simulation is dead and cuts off the force-feedback and the connection to the API.
For debugging purposes, it can be useful to disable the watchdog control, so that the simulation can be executed step-by-step without losing the connection with the Virtuose. In that case, it is essential to call the `virtClose` function at the end of the simulation, otherwise the haptic device will not be reset, and it could be impossible to establish a new connection.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `disable` parameter should be set to 1 to disable the watchdog control, and to 0 to re-enable it.

RETURN VALUE

In case of success, the `virtDisableControlConnexion` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtClose

NAME

virtSetPeriodicFunction – Time the simulation.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetPeriodicFunction (VirtContext VC,
                             void (*fn) (VirtContext, void*),
                             float* period,
                             void* arg);
```

DESCRIPTION

The `virtSetPeriodicFunction` function defines a call back function to be called at a fixed period of time, as timing for the simulation. The call back function is synchronized with messages received from the Virtuose controller, which arrive at very constant time intervals, and generate hardware interrupts not to be ignored by the operating system. In practise, this function is much more efficient for timing the simulation than common software timers.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `fn` parameter is a pointer to the call back function.
The `period` parameter corresponds the timing period, given in seconds. It should always be identical to the timestep given to the `virtSetTimeStep` function.
The `arg` parameter corresponds to an arbitrary parameter, to be used by the call back function.

RETURN VALUE

In case of success, the `virtSetPeriodicFunction` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtStartLoop

virtStopLoop

virtSetTimeStep

NAME

virtStartLoop – Start the simulation timing.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtStartLoop (VirtContext VC);
```

DESCRIPTION

The `virtStartLoop` function starts the periodic loop, which executes the call back function defined with `VirtSetPeriodicFunction` at a fixed rate.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

RETURN VALUE

In case of success, the `virtStartLoop` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetPeriodicFunction

virtStopLoop

NAME

virtStopLoop – Stop the simulation timing.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtStopLoop (VirtContext VC);
```

DESCRIPTION

The `virtStopLoop` function stops the periodic loop, which executes the call back function defined with `VirtSetPeriodicFunction` at a fixed rate.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

RETURN VALUE

In case of success, the `virtStopLoop` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetPeriodicFunction

virtStartLoop

NAME

virtSetSpeedFactor – Modify the movement scale factor.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetSpeedFactor(VirtContext VC, float factor);
```

DESCRIPTION

The `virtSetSpeedFactor` function modifies the speed factor, which corresponds to a scaling of the workspace of the haptic device.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `factor` parameter corresponds to the scale factor to be applied. A value larger than 1.0 means that the movements of the Virtuose are amplified inside the simulation.

RETURN VALUE

In case of success, the `virtSetSpeedFactor` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCORRECT_VALUE`
The value of the `factor` parameter is incorrect; this error is generated in the following cases:

- speed factor is negative or null
- speed factor is too large or too small to guarantee the stability of the system

SEE ALSO

virtGetSpeedFactor

virtSetForceFactor

NAME

virtGetSpeedFactor – Read the current value of the displacement scale factor.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetSpeedFactor(VirtContext VC, float *factor);
```

DESCRIPTION

The `virtGetSpeedFactor` function returns the current value of the speed factor, which corresponds to a scaling between movements of the VIRTUOSE and those in the simulation.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `factor` parameter is set by the function `virtGetSpeedFactor`. A value larger than 1 corresponds to an expansion of the workspace of the Virtuose.

RETURN VALUE

In case of success, the `virtGetSpeedFactor` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetSpeedFactor

virtGetForceFactor

NAME

virtSetForceFactor – Set the force scale factor.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetForceFactor(VirtContext VC, float factor);
```

DESCRIPTION

The `virtSetForceFactor` function sets the value of the force factor, which corresponds to a scaling between the forces exerted at the tip of the VIRTUOSE and those computed in the simulation.
The function must be called before the selection of the control mode.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `factor` parameter corresponds to the force scale. A value smaller than 1 corresponds to an amplification of the forces from the Virtuose towards the simulation.

RETURN VALUE

In case of success, the `virtSetForceFactor` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCORRECT_VALUE`
The value of the `factor` parameter is incorrect. This error is generated in the following cases:

- factor is negative or null
- speed factor too large or too small to guarantee the stability of the system

`VIRT_E_CALL_TIME`
The function is called after the selection of the control mode.

SEE ALSO

virtGetForceFactor

virtSetSpeedFactor

NAME

virtGetForceFactor – Read the current value of the force scale.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetForceFactor(VirtContext VC, float *factor);
```

DESCRIPTION

The `virtGetForceFactor` function returns the current value of the force factor, which corresponds to a scaling between the forces exerted at the tip of the VIRTUOSE and those computed in the simulation.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `factor` parameter is set by the function `virtGetForceFactor`. A factor smaller than the 1 corresponds to an amplification of the forces exerted by the user at the tip of the VIRTUOSE towards the simulation.

RETURN VALUE

In case of success, the `virtGetError` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetForceFactor,

virtGetSpeedFactor

NAME

virtSetTimeoutValue – Modify the communication timeout value.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetTimeoutValue(VirtContext VC, float tout);
```

DESCRIPTION

The `virtSetTimeoutValue` function modifies the timeout value used in communications with the Virtuose controller.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `tout` parameter corresponds to the new timeout value expressed in seconds.

RETURN VALUE

In case of success, the `virtSetTimeoutValue` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCORRECT_VALUE`
The value of `tout` parameter `is` incorrect.

SEE ALSO

virtGetTimeoutValue

NAME

virtGetTimeoutValue – Read the current timeout value.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetTimeoutValue(VirtContext VC, float *tout);
```

DESCRIPTION

The `virtGetTimeoutValue` function returns the current value of the timeout for the communication with the Virtuose controller.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `tout` parameter is set by the function with the current value of the timeout expressed in seconds.

RETURN VALUE

In case of success, the `virtGetTimeoutValue` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetTimeoutValue

NAME

virtSetBaseFrame – Set the Virtuose base frame into the observation frame.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetBaseFrame(VirtContext VC, float* base);
```

DESCRIPTION

The `virtSetBaseFrame` function sets the Virtuose base frame into the observation frame. It should be set only once, before starting the periodic update loop.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `base` parameter is an array of 7 floating point number. The first three values represent the translations and the four last values represent the rotation in a quaternion.

RETURN VALUE

In case of success, the `virtSetBaseFrame` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtGetBaseFrame

NAME

virtGetBaseFrame – Read the current position of the base reference frame.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetBaseFrame(VirtContext VC, float *pos);
```

DESCRIPTION

The `virtGetBaseFrame` function returns the current position of the base reference frame with respect to the observation reference frame.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `pos` parameter corresponds to the transformation from the observation reference frame to the base reference frame.

RETURN VALUE

In case of success, the `virtGetBaseFrame` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetBaseFrame

NAME

virtSetIndexingMode – Modify the mode of indexing.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetIndexingMode(VirtContext VC, VirtIndexingType mode);
```

DESCRIPTION

The `virtSetIndexingMode` function changes the mode of indexing (also called offset).

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `mode` parameter corresponds to the mode of indexing. The different values are:

`INDEXING_ALL (0)`
Indexing is active for both translation and rotation movements, whenever the offset button is pushed or the power is off (power button or deadman sensor off).

`INDEXING_TRANS (1)`
Indexing is active only on the translation movements.

`INDEXING_NONE (2)`
Indexing is inactive (the offset button is inactive).

`INDEXING_ALL_FORCE_FEEDBACK_INHIBITION (3)`
Indexing is active for both translation and rotation movements, whenever the offset button is pushed or the power is off (power button or deadman sensor off), and the force-feedback is disabled while indexing is active.

`INDEXING_TRANS_FORCE_FEEDBACK_INHIBITION (4)`
Indexing is active only on the translation movements. No force-feedback during indexing.

`INDEXING_ROT_FORCE_FEEDBACK_INHIBITION (6)`
Indexing is active only on the rotation movements. No force-feedback during indexing.

`INDEXING_ROT (7)`
Indexing is active only on the rotation movements.

RETURN VALUE

In case of success, the `virtSetIndexingMode` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCORRECT_VALUE`
The value of `mode` parameter does not correspond to a valid mod.

NAME

virtGetIndexingMode – Read the current indexing mode.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetIndexingMode (VirtContext VC, VirtIndexingMode *mode);
```

DESCRIPTION

The `virtGetIndexingMode` function returns the current indexing mode (also called offset mode).

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `mode` parameter is set by the function. The possible values are:

`INDEXING_ALL (0)`

`INDEXING_TRANS (1)`

`INDEXING_NONE (2)`

`INDEXING_ALL_FORCE_FEEDBACK_INHIBITION (3)`

`INDEXING_TRANS_FORCE_FEEDBACK_INHIBITION (4)`

`INDEXING_ROT_FORCE_FEEDBACK_INHIBITION (6)`

`INDEXING_ROT (7)`

RETURN VALUE

In case of success, the `virtGetIndexingMode` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetIndexingMode

NAME

virtSetPowerOn – Software authorization to set the power on.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetPowerOn(VirtContext VC, int);
```

DESCRIPTION

The `virtSetPowerOn` function gives the software authorization to set the motors under power.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `power` parameter corresponds to the software authorization. The value 1 means the authorization to set the motors under power is granted, the value 0 means the prohibition to the motors under power.

RETURN VALUE

In case of success, the `virtSetPowerOn` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtGetPowerOn

NAME

virtSetCommandType – Set the desired control mode.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetCommandType(VirtContext VC, VirtCommandType type);
```

DESCRIPTION

The `virtSetCommandType` function changes the control mode of the Virtuose device.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `type` parameter corresponds to the desired control mode. The possible values are:

`COMMAND_TYPE_NONE`
No possible movement

`COMMAND_TYPE_IMPEDANCE`
Force/position control

`COMMAND_TYPE_VIRTMECH`
Position/force control with virtual mechanism

`COMMAND_TYPE_ARTICULAR`
Articular position control

`COMMAND_TYPE_ARTICULAR_IMPEDANCE`
Articular force control

RETURN VALUE

In case of success, the `virtSetCommandType` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

`VIRT_E_INCORRECT_VALUE`
The value of `type` parameter does not correspond to a valid control mode.

SEE ALSO

virtGetCommandType

NAME

virtGetCommandType – Read the current control mode.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetCommandType(VirtContext VC, VirtCommandType *type);
```

DESCRIPTION

The `virtGetCommandType` function returns the current control mode.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `type` parameter is set by the function.
Possible values are:

`COMMAND_TYPE_NONE`
No possible movement

`COMMAND_TYPE_IMPEDANCE`
Force/position control

`COMMAND_TYPE_VIRTMECH`
Position/force control with virtual mechanism

`COMMAND_TYPE_ARTICULAR`
Articular position control

`COMMAND_TYPE_ARTICULAR_IMPEDANCE`
Articular force control

RETURN VALUE

In case of success, the `virtGetCommandType` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetCommandType

NAME

virtSetDebugFlags – Set the level of diagnosis information output.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetDebugFlags(VirtContext VC, unsigned short flags);
```

DESCRIPTION

The `virtSetDebugFlags` function sets the level of diagnosis information output generated by the VIRTUOSE API. The debug information is showed on the console, or written in a file if one was specified with `virtSetoutputFile`.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `flags` parameter is a field of bits corresponding to the various levels of diagnosis to activate:

- DEBUG_SERVO: information related to the system control stability
- DEBUG_LOOP: information related to the simulation timing

RETURN VALUE

In case of success, the `virtSetDebugFlags` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

SEE ALSO

NAME

virtGetDeviceID – read device type and serial number.

SYNOPSYS

```
#include "virtuoseAPI.h"
int virtGetDeviceID(VirtContext VC,
                    int * device_type
                    int * serial_number);
```

DESCRIPTION

The `virtGetDeviceID` function reads the device type and serial number stored in its embedded variator card.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `device_type` parameter corresponds to the device type identifier in the HAPTION product line.

```
DEVICE_VIRTUOSE_3D              1
DEVICE_VIRTUOSE_3D_DESKTOP      2
DEVICE_VIRTUOSE_6D              3
DEVICE_VIRTUOSE_6D_DESKTOP      4
DEVICE_VIRTUOSE_7D              5
DEVICE_MAT6D                    6
DEVICE_MAT7D                    7
DEVICE_INCA_6D                  8
DEVICE_INCA_3D                  9
DEVICE_ORTHESE                  10
DEVICE_SCALE1                   11
DEVICE_1AXE                     12
DEVICE_OTHER                    13
```

The `serial_number` parameter corresponds to the device serial number.

RETURN VALUE

In case of success, the `virtGetDeviceID` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

`VIRT_E_INCOMPATIBLE_VERSION`
The embedded controller version is below 5.50.

SEE ALSO

NAME

virtSetGripperCommandType – Choose the command type of the pinch.

SYNOPSYS

```
#include "virtuoseAPI.h"
int virtSetGripperCommandType(VirtContext VC,
                              VirtGripperCommandType type);
```

DESCRIPTION

The function set the command type of the pinch when it is connected on the 7th axis of the variator card.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `type` parameter corresponds to the command type:

```
GRIPPER_COMMAND_TYPE_NONE : pinch not used
GRIPPER_COMMAND_TYPE_POSITION : pinch used in position mode
GRIPPER_COMMAND_TYPE_IMPEDANCE : pinch used in impedance mode
```

RETURN VALUE

In case of success, the `virtGetDeviceID` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

SEE ALSO

NAME

virtSetTimeStep – Set the value of the simulation timestep.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetTimeStep(VirtContext VC, float timeStep);
```

DESCRIPTION

The `virtSetTimeStep` function informs the Virtuose controller of the simulation timestep. This value is used in order to guarantee the stability of the system.
The function must be called before the selection of the type of control mode.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `timeStep` parameter corresponds to the simulation timestep, expressed in seconds.

RETURN VALUE

In case of success, the `virtGetError` function returns `0`. Otherwise, it returns `-1` and the function `virtGetErrorCode` gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`

The `VC` parameter does not correspond to a valid `VirtContext` object

`VIRT_E_INCORRECT_VALUE`
The value of the parameter `timeStep` is incorrect. This error is generated in the following cases:

- negative or null value
- value lower than the timestep of the controller
- value too large to ensure the stability of the system

`VIRT_E_CALL_TIME`
The function is called after the selection of the type of control mode.

SEE ALSO

virtGetTimeStep

NAME

virtGetTimeStep – Read the current simulation timestep.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetTimeStep(VirtContext VC, float* step);
```

DESCRIPTION

The `virtGetTimeStep` function returns the value of the simulation timestep given beforehand by the function `virtGetTimeStep`. It returns zero if not defined.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `step` parameter is set by the function, and corresponds of the simulation timestep expressed in seconds.

RETURS VALUE

The `virtGetTimeStep` function returns `0` in all the cases.

ERRORS

`VIRT_E_INVALID_CONTEXT`

The `VC` parameter does not correspond to a valid `VirtContext` object

SEE ALSO

virtSetTimeStep

5.4 Object attachment

NAME

virtAttachVO – Attach the VIRTUOSE to an object.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtAttachVO(VirtContext VC, float mass, float *inertia);
```

DESCRIPTION

The `virtAttachVO` function implements an attachment of an object to the VIRTUOSE, and computes optimal control parameters while garanteeing the global system stability. The object is defined by the position of its centre (reduction point of the force tensor), which has to be provided by using the `virtSetPosition` function after calling `virtAttachVO`.
This function is only accessible in control modes `COMMAND_TYPE_VIRTMECH`.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `mass` parameter corresponds to the mass of the object, expressed in kg.
The `inertia` parameter corresponds to the inertia matrix of the object, stored in lines as a vector with 9 values.

RETURN VALUE

In case of success, the `virtAttachVO` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtDetachVO

virtSetPosition

virtSetSpeed

virtGetForce

virtAttachVOAvatar

virtAttachQSVO

NAME

virtDetachVO – Detach an object from the VIRTUOSE.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtDetachVO(VirtContext VC);
```

DESCRIPTION

The `virtDetachVO` function cancels the attachment of an object with the VIRTUOSE, and set all control parameters to zero.
This function is only accessible in control modes `COMMAND_TYPE_VIRTMECH` and `COMMAND_TYPE_IMPEDANCE`.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

RETURN VALUE

In case of success, the `virtDetachVO` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtAttachVO

NAME

virtAttachVOAvatar – Attach the VIRTUOSE to an object.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtAttachVOAvatar(VirtContext VC,
                       float mass, float *inertia);
```

DESCRIPTION

The `virtAttachVOAvatar` function implements an attachment of an object to the VIRTUOSE, and computes optimal control parameters while garanteeing the global system stability.
The object is defined by the position of its centre (reduction point of the force tensor), which has to be provided by using the `virtSetPosition` function before calling virtAttachVOAvatar.
The difference with the `virtAttachVO` function is the attachment of the object in its gravity centre in the avatar frame. Therefore, there is a lever arm.
This function is only accessible in the control mode `COMMAND_TYPE_VIRTMECH` and `COMMAND_TYPE_IMPEDANCE`.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `mass` parameter corresponds to the mass of the object, expressed in kg.
The `inertia` parameter corresponds to the inertia matrix of the object, stored in lines as a vector with 9 values.

RETURN VALUE

In case of success, the `virtAttachVOAvatar` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtDetachVOAvatar

virtSetCatchFrame

NAME

virtDetachVOAvatar – Detach an object from the VIRTUOSE.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtDetachVOAvatar(VirtContext VC);
```

DESCRIPTION

The `virtDetachVOAvatar` function cancels the attachment of an object with the VIRTUOSE, and set all control parameters to zero.
This function is only accessible in control modes `COMMAND_TYPE_VIRTMECH` and `COMMAND_TYPE_IMPEDANCE`.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

RETURN VALUE

In case of success, the `virtDetachVOavatar` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtAttachVOAvatar

virtAttachQSVO

NAME

virtSetCatchFrame – Modify the attachment point of a virtual object.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetCatchFrame (VirtContext VC, float* frame);
```

DESCRIPTION

The `virtSetCatchFrame` function specifies the position of the point where an object can be attached to the Virtuose, also called “catch frame”. The catch frame has to be defined before calling `virtAttachVO`. It is reset to identity after each call to `virtDettachVO`. The rotation part of the catch frame position is not taken into account.
This function is only available in control mode `COMMAND_TYPE_VIRTMECH`.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `frame` parameter corresponds to the transformation from the centre of the virtual object to the centre of the catch frame, expressed as a displacement vector of 7 components, only the first 3 of which are used.

RETURN VALUE

In case of success, the `virtSetCatchFrame` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

`VIRT_E_INCORRECT_VALUE`
The value of the `frame` parameter is invalid.

SEE ALSO

virtAttachVO

virtDettachVO

virtGetCatchFrame

NAME

virtGetCatchFrame – Read the current position of the attachment point.

SYNOPTIQUE

```
#include "virtuoseAPI.h"
int virtGetCatchFrame (VirtContext VC, float* frame);
```

DESCRIPTION

The `virtGetCatchFrame` function returns the current value of the catch frame, previously specified by `virtSetCatchFrame`.
The function is available only in control mode `COMMAND_TYPE_VIRTMECH`.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The f`rame` parameter corresponds to the transformation from the centre of the virtual object to the centre of the catch frame, expressed as a displacement vector of 7 components, only the first 3 of which are used.

RETURN VALUE

In case of success, the `virtGetCatchFrame` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

SEE ALSO

virtAttachVO

virtDettachVO

virtSetCatchFrame

5.5 Virtuose state access

NAME

virtGetButton – Read the state of a push-button.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetButton(VirtContext VC, int button, int *state);
```

DESCRIPTION

The `virtGetButton` function returns the state of one of the push-buttons situated on the tool placed at the tip of the Virtuose. Only three buttons are supported, and their respective index depends on the type of Virtuose product.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `button` parameter corresponds of the button index.
The `state` parameter is set by the function (0: button released, 1: button pushed).

RETURN VALUE

In case of success, the `virtGetButton` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtGetDeadMan

virtGetEmergencyStop

NAME

virtGetDeadMan – Read the state of the safety sensor.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetDeadMan(VirtContext VC, int *deadman);
```

DESCRIPTION

The `virtGetDeadMan` function returns the state of the safety sensor placed at the end of the Virtuose (also called "dead man sensor").

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `deadman` parameter is set by the function. A value of 1 means that the safety sensor is active (user present), a value of 0 means that the sensor is inactive (user absent).

RETURN VALUE

In case of success, the `virtGetDeadMan` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtGetEmergencyStop

virtGetPowerOn

NAME

virtGetEmergencyStop – Read the state of the emergency stop.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetEmergencyStop(VirtContext VC, int *stop);
```

DESCRIPTION

The `virtGetEmergencyStop` function returns the state of the emergency stop.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `stop` parameter is set by the function. A value of 1 means that the chain is closed (the system is operational), a value of that it is open (the system is stopped).

RETURN VALUE

In case of success, the `virtGetEmergencyStop` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.
.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtGetDeadMan

virtGetPowerOn

NAME

virtGetError – Operational status of the Virtuose.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetError (VirtContext VC, int* error);
```

DESCRIPTION

The `virtGetError` function tests the operational status of the Virtuose, in terms of breakdowns or simple errors at the level of each material component. The components tested are the amplifiers and the position encoders.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `error` parameter is set by the function. Value 0 means that the Virtuose is in good operational conditions. Value 1 means breakdown or error of operation.

RETURN VALUE

In case of success, the `virtGetError` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

VIRT_E_HARDWARE_ERROR
This error message means the presence of one or more breakdowns or errors.

SEE ALSO

virtDisplayHardwareStatus

NAME

virtGetErrorCode – Get the code of the last error encountered.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetErrorCode(VirtContext VC);
```

DESCRIPTION

The `virtGetErrorCode` function returns the code of the last error encountered on the connection with the Virtuose controller corresponding to `VC` parameter.

PARAMETERS

The `VC` parameter corresponds to the VirtContext object returned by the `virtOpen` function, or `NULL` in the case of an error in the call to `virtOpen`.

RETURN VALUE

By definition, the `virtGetErrorCode` function always returns an error code (see below).

ERRORS

The errors below have a generic significance. Sometimes, the diagnostic depends on the function which signalled an error. Refer then to the description of the function.

`VIRT_E_NO_ERROR`
No error encountered.

`VIRT_E_OUT_OF_MEMORY`
Memory allocation error.

`VIRT_E_INVALID_CONTEXT`
An invalid `VirtContext` parameter was given to the API.

`VIRT_E_COMMUNICATION_FAILURE`
An error was encountered in the communication with the controller of the Virtuose.

`VIRT_E_FILE_NOT_FOUND`
The file given as parameter does not exist.

`VIRT_E_WRONG_FORMAT`
An error was detected in a format of data given to the API.

`VIRT_E_TIME_OUT`
The maximum time was exceeded in waiting for an answer from the controller of the Virtuose.

`VIRT_E_NOT_IMPLEMENTED`
The function is not implemented in this version of the API.

`VIRT_E_VARIABLE_NOT_AVAILABLE`
The required variable is not available on this model of Virtuose.

`VIRT_E_INCORRECT_VALUE`
A value transmitted to API is incorrect or outside of authorized bounds.

`VIRT_E_HARDWARE_ERROR`
A failure was detected in the operation of one hardware component of the Virtuose.

SEE ALSO

virtOpen

virtGetErrorMsg

virtGetFailure

NAME

virtGetErrorMessage – Get a description of an error code.

SYNOPSIS

```
#include "virtuoseAPI.h"
const char *virtGetErrorMessage(int code);
```

DESCRIPTION

The `virtGetErrorMessage` function returns a message describing the error code given as parameter.

PARAMETERS

The `code` parameter corresponds to the error code.

RETURN VALUE

In all the cases, the `virtGetErrorMessage` function returns an error message.

ERRORS

None

SEE ALSO

virtGetErrorCode

NAME

virtGetFailure – Get encoder failure code.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetFailure(VirtContext VC, unsigned int *error);
```

DESCRIPTION

The `virtGetFailure` function returns the encoder failure code. It should be used in complement with the `virtGetErrorCode` function.

PARAMETERS

The `VC` parameter corresponds to the `VirtContext` object returned by the `virtOpen` function, or `NULL` in the case of an error in the call to `virtOpen`.
The `error` parameter corresponds to failure code returned by the function.

RETURN VALUE

In case of success, the `virtGetFailure` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtGetErrorCode

NAME

virtDisplayHardwareStatus – Hardware-level failure diagnosis.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtDisplayHardwareStatus (VirtContext VC, FILE *file);
```

DESCRIPTION

The `virtDisplayHardwareStatus` function returns a state vector of the VIRTUOSE hardware-level, in order to diagnose breakdowns and errors of each material component. The components considered are the amplifiers and the position encoders.

PARAMETERS

The `VC` parameter corresponds to the object of type `VirtContext` returned by the function `virtOpen`.
The `file` parameter is currently not used. The output device is defined by the `virtSetOutputFile` function (or stdout by default).

RETURN VALUE

In case of success, the `virtDisplayHardwareStatus` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

VirtSetOutputFile

virtGetError

NAME

virtGetPowerOn – Read the state of the motor power supply.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetPowerOn(VirtContext VC, int *poweron);
```

DESCRIPTION

The `virtGetPowerOn` function returns the state of the motor power supply.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `poweron` parameter is set by the function. A value of 1 means that the motors are under power, a value 0 that they are not.

RETURN VALUE

In case of success, the `virtGetPowerOn` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetPowerOn

virtGetEmergencyStop

virtGetDeadMan

NAME

virtIsInBounds – Test whether the mechanical limits of the device workspace have been reached.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtIsInBounds (VirtContext VC, unsigned int *bounds);
```

DESCRIPTION

The `VirtIsInBounds` function returns a bit field giving the status of all mechanical limits of the device workspace.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `bounds` parameter is set by the function, as a field of bit with the following meaning:
`VIRT_BOUND_LEFT_AXE_1` corresponds to the left bound of axis 1,
`VIRT_BOUND_RIGHT_AXE_1` corresponds to the right bound of axis 1,
`VIRT_BOUND_SUP_AXE_2` corresponds to the upper bound of axis 2,
`VIRT_BOUND_INF_AXE_2` corresponds to the lower bound of axis 2,
`VIRT_BOUND_SUP_AXE_3` corresponds to the upper bound of axis 3,
`VIRT_BOUND_INF_AXE_3` corresponds to the lower bound of axis 3,
`VIRT_BOUND_LEFT_AXE_4` corresponds to the left bound of axis 4,
`VIRT_BOUND_RIGHT_AXE_4` corresponds to the right bound of axis 4,
`VIRT_BOUND_SUP_AXE_5` corresponds to the upper bound of axis 5,
`VIRT_BOUND_INF_AXE_5` corresponds to the lower bound of axis 5,
`VIRT_BOUND_LEFT_AXE_6` corresponds to the left bound of axis 6,
`VIRT_BOUND_RIGHT_AXE_6` corresponds to the right bound of axis 6.

RETURN VALUE

In case of success, the `virtIsInBounds` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

NAME

virtGetAlarm – Read the current alarm status.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetAlarm (VirtContext VC, unsigned int *alarm);
```

DESCRIPTION

The `virtGetAlarm` function returns the current alarm status.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `alarm` parameter is set by the function, as a field of bits with the following meaning:

- `VIRT_ALARM_OVERHEAT` means that one motor is overheated. Force-feedback is automatically reduced, until the motor has cooled down to an acceptable temperature.
- `VIRT_ALARM_SATURATE` means that the motor currents have reached their maximum and are saturated.
- `VIRT_ALARM_CALLBACK_OVERRUN` means that the execution time of the call back function defined with `virtSetPeriodicFunction` is greater than the timestep value. In that case, the real-time execution of the simulation cannot be guaranteed.

RETURN VALUE

In case of success, the `virtGetAlarm` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

SEE ALSO

virtSaturateTorque

virtSetPeriodicFunction

virtSetTimeStep

NAME

virtIsInShiftPosition – Test the status of indexing.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtIsInShiftPosition(VirtContext VC, int* indexing);
```

DESCRIPTION

The `virtIsInShiftPosition` function returns the current state of indexing, that is:

- a value of 1 if the offset push-button is pressed or the power if off
- a value of 0 otherwise.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

The `indexing` parameter corresponds to the state of indexing.

RETURN VALUE

In case of success, the `virtIsInShiftPosition` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`

The `VC` parameter does not correspond to a valid `VirtContext` object

SEE ALSO

virtSetIndexingMode

virtGetIndexingMode

virtGetPowerOn

NAME

virtGetMouseState – Read the state of push-buttons in mouse mode.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetMouseState(VirtContext VC,
 ......               int* active,
 ......               int* left,
 ......               int* right);
```

DESCRIPTION

This function is obsolete, and does nothing.

PARAMETERS

RETURN VALUE

The `virtGetMouseState` function `always` returns `0`.

ERRORS

SEE ALSO

NAME

virtGetTrackball – Read the trackball.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetTrackball(VirtContext VC, int* x_move, int* y_move);
```

DESCRIPTION

This function is obsolete, and does nothing.

PARAMETERS

RETURN VALUE

The `virtGetTrackball` function `always` returns `0`.

ERRORS

SEE ALSO

NAME

virtGetTrackballButton – Read the buttons of the trackball.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetTrackballButton(VirtContext VC,
 ......                    int* actif,
 ......                    int* btn_gauche,
 ......                    int* btn_milieu,
 ......                    int* btn_droit);
```

DESCRIPTION

This function is obsolete, and does nothing.

PARAMETERS

RETURN VALUE

The `virtGetTrackballButton` function `always` returns `0`.

ERRORS

SEE ALSO

NAME

virtGetADC – Read an analogic input.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetADC(VirtContext VC, int line, float* adc);
```

DESCRIPTION

The `virtGetADC` function reads one of the first three analogic inputs after applying a gain, an offset and filtering. It can be used if the haptic device is a virtuose 3D Desktop, for the orientation of the handle.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `line` parameter corresponds to the ADC input to read. It can only take the value 1, 2 or 3, no more analogic inputs are available.
The `adc` parameter corresponds to the read value of the ADC.

RETURN VALUE

In case of success, the `virtGetADC` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtGetAnalogicInputs

NAME

virtGetAnalogicInputs – Get all analogic inputs

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetAnalogicInputs (VirtContext VC, float* inputs);
```

DESCRIPTION

The `virtGetAnalogicInputs` function returns the raw filtered value of the analogic inputs of the device. For example, the trigger of the TREH handle or the turntable. Many hactic device haven't this type of inputs.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `inputs` parameter is a table of six floating value. It is sets by the API to the current inputs value.

RETURN VALUE

In case of success, the `virtGetAnalogicInputs` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCOMPATIBLE_VERSION`
The embedded software version is lower than 5.59.

SEE ALSO

virtGetADC

NAME

virtWaitPressButton – Wait for user to press the button.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtWaitPressButton(VirtContext VC,
                        int button_number);
```

DESCRIPTION

The `virtWaitPressButton` function waits for the user to press the button. It is a blocking function. The function falls asleep after receiving the signal of the pressed button (number given in parameter). Only three buttons are supported, and the index of the buttons depend on the type of Virtuose device.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The b`utton_number` parameter corresponds to the button number.

RETURN VALUE

In case of success, the `virtWaitPressButton` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

NAME

virtForceShiftButton – Force the clutching.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtForceShiftButton(VirtContext VC, int forceShiftButton);
```

DESCRIPTION

The `virtForceShiftButton` function forces the clutching. It is a software way like pressing the clutch button.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `forceShiftButton` parameter is an integer. The value 1 means to force the clutching, otherwise the value 0 means to deactivate the forced clutching.

RETURN VALUE

In case of success, the `virtForceShiftButton` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

NAME

virtGetTimeLastUpdate – Get the time elapsed since the last update of the Virtuose state vector.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetTimeLastUpdate (VirtContext, unsigned int *tout);
```

DESCRIPTION

The `virtGetTimeLastUpdate` function returns the time elapsed since the last update of the Virtuose state vector.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `tout` parameter is set by the function with the value in CPU ticks of the time elapsed since the last update of the state vector received from the Virtuose controller.

RETURN VALUE

In case of success, the `virtGetTimeLastUpdate` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

SEE ALSO

NAME

virtGetNbAxes – Get the number of joints

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetNbAxes(VirtContext VC, int* nbAxes);
```

DESCRIPTION

The `virtGetNbAxes` function returns the number of controlled joints of the Virtuose.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `nbAxes` parameter is a pointer. The value is set by the function with the number of joints of the Virtuose that can be controlled.

RETURN VALUE

In case of success, the `virtGetNbAxes` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

NAME

virtGetDigitalInputs – Get digital inputs values

SYNOPSIS

```
#include "virtuoseAPI.h"

#define virtGetDigitalInputs virtGetTorInputs

int virtGetTorInputs(VirtContext VC, unsigned int *tor_in);
```

DESCRIPTION

The `virtGetDigitalInputs` function returns the value of the digital inputs of the Virtuose. This function is used to get the status of digital inputs:

- power button
- deadman

The value depends on the haptic device and the type of handle.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `tor_in` parameter is an unsigned integer which represents a bit-field. It is set by the function with the digital values.

RETURN VALUE

In case of success, the `virtGetTorInputs` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCOMPATIBLE_VERSION`
The embedded software version is lower than 5.59.

SEE ALSO

5.6 Virtuose control

NAME

virtSetPosition – Modify the current control position.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetPosition(VirtContext VC, float *pos_p_env);
```

DESCRIPTION

The `virtSetPosition` function modifies the current value of the control position and sends it to the Virtuose controller. If an object is attached to the Virtuose (`virtAttachVO` called before), then the control point is the centre of the object, otherwise it is the centre of the Virtuose end-effector.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `pos_p_env` parameter is a displacement vector of 7 components (see glossary).

RETURN VALUE

In case of success, the `virtSetPosition` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCORRECT_VALUE`
The value of the `pos_p_env` parameter is incorrect. This error is generated in the following case:

- The module of the quaternion part is different from 1.0.

SEE ALSO

virtAttachVO

virtSetSpeed

NAME

virtGetPosition – Read the current position of the VIRTUOSE, or of the object attached to the VIRTUOSE.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetPosition(VirtContext VC, float *pos_p_env);
```

DESCRIPTION

The `virtGetPosition` function returns the current position of the tool frame of the VIRTUOSE, or of the object attached to the VIRTUOSE if the `virtAttachVO` function has been called before.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `pos_p_env` parameter is set by the `virtGetPosition` function. It is expressed in the form of a displacement vector with 7 components, in the coordinates of the environment reference frame.

RETURN VALUE

In case of success, the `virtGetPosition` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetPosition

virtAttachVO

NAME

virtSetSpeed – Modify the current control speed.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetSpeed(VirtContext VC, float *speed_p_env_r_ps);
```

DESCRIPTION

The `virtSetSpeed` function modifies the current value of the control speed. If an object is attached to the VIRTUOSE (`virtAttachVO` called before), then the control point is the centre of the object, otherwise it is the centre of the VIRTUOSE end-effector.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `speed_p_env_r_ps` parameter corresponds to the speed with respect to the control point expressed in the coordinates of the environment reference frame.

RETURN VALUE

In case of success, the `virtSetSpeed` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtAttachVO,

virtSetPosition

NAME

virtGetSpeed – Read the speed of the VIRTUOSE, or of the object attached to the VIRTUOSE.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetSpeed(VirtContext VC, float *speed_p_env_r_ps);
```

DESCRIPTION

The `virtGetSpeed` function returns the speed of the VIRTUOSE, or of the object attached to the VIRTUOSE if the `virtAttachVO` function was called before.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `speed_p_env_r_ps` parameter is set by the function `virtGetSpeed`. It is expressed in the form of a cinematic tensor with 6 components, giving the speed of the Virtuose (or of the object attached to it) with respect to tip of the Virtuose (or to the centre of the object attached to it) in the coordinates of the environment reference frame.

RETURN VALUE

In case of success, the `virtGetSpeed` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetSpeed

virtAttachVO

NAME

virtSetForce – Send the force to be applied to the VIRTUOSE.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetForce(VirtContext VC, float *force_p_env_r_ps);
```

DESCRIPTION

The `virtSetForce` function sends the force to be applied to the VIRTUOSE, in the case of a force/position control (impedance control).

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `force_p_env_r_ps` parameter contains the force to be applied. It is expressed in the form of a dynamic tensor with 6 components, with respect to the Virtuose end-effector and expressed in the coordinates of the environment reference frame.
This function is accessible only in `COMMAND_TYPE_IMPEDANCE` control mode.

RETURN VALUE

In case of success, the `virtSetForce` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetCommandType

NAME

virtGetForce – Return the force tensor to be applied to the attached object.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetForce(VirtContext VC, float *force_p_env_r_ps);
```

DESCRIPTION

The `virtGetForce` function returns the force tensor to be applied to the object attached to the VIRTUOSE, allowing the dynamic simulation of the scene.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `force_p_env_r_ps` parameter is set by the `virtGetForce` function. It corresponds to the force applied by the user to the Virtuose, in the form of a 6-component force tensor. This force is expressed with respect to the centre of the object, in the coordinates of the environment reference frame.

RETURN VALUE

In case of success, the `virtGetError` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetPosition

virtSetSpeed,

virtAttachVO

NAME

virtSetObservationFrame – Move the observation frame.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetObservationFrame(VirtContext VC, float *pos);
```

DESCRIPTION

The `virtSetObservationFrame` function modifies the position of the observation frame with respect to the reference of environment.
It can be called at any time.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `pos` parameter corresponds to the transformation from the environment reference frame to the observation reference frame, expressed as a displacement vector of 7 components.

RETURN VALUE

In case of success, the `virtSetObservationFrame` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCORRECT_VALUE`
The `pos` parameter is not a valid displacement vector.

`VIRT_E_CALL_TIME`
The function is called after the command type selection.

SEE ALSO

VirtGetObservationFrame

VirtSetObservationFrameSpeed

NAME

virtGetObservationFrame – Read the current position of the observation reference frame.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetObservationFrame(VirtContext VC, float *pos);
```

DESCRIPTION

The `virtGetObservationFrame` function returns the current position of the observation reference frame with respect to the environment reference frame.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `pos` parameter corresponds to the transformation from the environment reference frame to the observation reference frame.

RETURN VALUE

In case of success, the `virtGetObservationFrame` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetObservationFrame

NAME

virtSetObservationFrameSpeed – Modify the speed of the observation reference frame.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetObservationFrameSpeed(VirtContext VC, float *speed);
```

DESCRIPTION

The `virtSetObservationFrameSpeed` function sets the speed of the observation reference frame with respect to the environment reference frame.
It can be called at any time.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `speed` parameter corresponds to the speed of motion of the observation reference frame with respect to the environment reference frame.

RETURN VALUE

In case of success, the `virtSetObservationFrameSpeed` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtGetObservationFrame

VirtSetObservationFrame

NAME

virtGetAvatarPosition – Read the indexed position of the end-effector.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetAvatarPosition (VirtContext VC, float* pos);
```

DESCRIPTION

The `VirtGetAvatarPosition` function returns the current position of the tool frame, taking into account the current offsets (or indexing) and the motion scale factor.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `pos` parameter corresponds to the position. It is expressed as a displacement vector of 7 components, in the coordinates of the environment reference frame. In case no object has been attached to the Virtuose (no previous call to `virtAttachVO`), the function returns the same result as `virtGetPosition`.

RETURN VALUE

In case of success, the `virtGetAvatarPosition` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtGetAvatarSpeed

virtSetSpeedFactor

virtGetPosition

NAME

virtGetPhysicalPosition – Read the physical position of the haptic device.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetPhysicalPosition (VirtContext VC, float* pos);
```

DESCRIPTION

The `virtGetPhysicalPosition` function returns the physical position of the Virtuose with respect to its base.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `pos` parameter corresponds to the position. It is expressed as a 7 floating-points array in the coordinates of the base reference frame of the Virtuose.

RETURN VALUE

In case of success, the `virtGetPhysicalPosition` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetBaseFrame

virtGetPhysicalSpeed

NAME

virtGetPhysicalSpeed – Read the physical speed of the haptic device.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetPhysicalSpeed (VirtContext VC, float* speed);
```

DESCRIPTION

The `virtGetPhysicalSpeed` function returns the physical position of the Virtuose with respect to its base.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `speed` parameter corresponds to the speed. It is expressed in the coordinates of the base reference frame of the Virtuose. The first 3 values represent to the translation speed (Vx, Vy, Vz) and the last 3 to the rotation speed (VRx, VRy, VRz).

RETURN VALUE

In case of success, the `virtGetPhysicalSpeed` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetBaseFrame

virtGetPhysicalPosition

NAME

virtAddForce – Add a force to the VIRTUOSE.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtAddForce(VirtContext VC, float *force_p_bm_r_pm);
```

DESCRIPTION

The `virtAddForce` function adds a force to be applied to the VIRTUOSE.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `force_p_bm_r_pm` parameter contains the force to be applied. It is expressed in the form of a dynamic tensor with 6 components, with respect to the Virtuose end-effector and expressed in the coordinates of the base frame.

RETURN VALUE

In case of success, the `virtAddForce` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

NAME

virtGetArticularPositionOfAdditionalAxe – Read articular position of the gripper.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetArticularPositionOfAdditionalAxe(VirtContext VC,
 ..............................              float* pos);
```

DESCRIPTION

The `virtGetArticularPositionOfAdditionalAxe` function reads the articular position of an additional motor or of the gripper.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `pos` parameter corresponds to the position.

RETRUN VALUE

In case of success, the `virtGetArticularPositionOfAdditionalAxe` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetArticularPositionOfAdditionalAxe

virtGetArticularSpeedOfAdditionalAxe

virtSetArticularSpeedOfAdditionalAxe

virtSetArticularForceOfAdditionalAxe

NAME

virtSetArticularPositionOfAdditionalAxe – Send articular position order.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetArticularPositionOfAdditionalAxe(VirtContext VC,
 ..............................             float* pos);
```

DESCRIPTION

The virtSetArticularPositionOfAdditionalAxe function sends an articular position order to an additional motor or to the gripper.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `pos` parameter corresponds to the position.

RETURN VALUE

In case of success, the `virtGetArticularPositionOfAdditionalAxe` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtGetArticularPositionOfAdditionalAxe

virtGetArticularSpeedOfAdditionalAxe

virtSetArticularSpeedOfAdditionalAxe

virtSetArticularForceOfAdditionalAxe

NAME

virtGetArticularSpeedOfAdditionalAxe – Read articular speed of the gripper.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetArticularSpeedOfAdditionalAxe(VirtContext VC,
 ..............................          float* pos);
```

DESCRIPTION

The `virtGetArticularSpeedOfAdditionalAxe` function read the articular speed of an additional motor or of the gripper.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `speed` parameter corresponds to the speed.

RETURN VALUE

In case of success, the `virtGetArticularSpeedOfAdditionalAxe` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

SEE ALSO

virtSetArticularSpeedOfAdditionalAxe

virtGetArticularPositionOfAdditionalAxe

virtSetArticularPositionOfAdditionalAxe

virtSetArticularForceOfAdditionalAxe

NAME

virtSetArticularSpeedOfAdditionalAxe – Send articular speed order.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetArticularSpeedOfAdditionalAxe(VirtContext VC,
 ..............................          float* pos);
```

DESCRIPTION

The `virtSetArticularSpeedOfAdditionalAxe` function an articular speed order to an additional motor or to the gripper.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `speed` parameter corresponds to the speed.

RETURN VALUE

In case of success, the `virtSetArticularSpeedOfAdditionalAxe` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtGetArticularSpeedOfAdditionalAxe

virtGetArticularPositionOfAdditionalAxe

virtSetArticularPositionOfAdditionalAxe

virtSetArticularForceOfAdditionalAxe

NAME

virtSetArticularForceOfAdditionalAxe – Send articular force order.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetArticularForceOfAdditionalAxe(VirtContext VC,
 ..............................          float* effort);
```

DESCRIPTION

The `virtSetArticularForceOfAdditionalAxe` function sends an articular force order to an additional motor or to the gripper.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `effort` parameter corresponds to the articular force.

RETURN VALUE

In case of success, the `virtSetArticularForceOfAdditionalAxe` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtGetArticularPositionOfAdditionalAxe

virtGetArticularPositionOfAdditionalAxe

virtSetArticularSpeedOfAdditionalAxe

virtGetArticularSpeedOfAdditionalAxe

NAME

virtGetArticularPosition – Read articular position.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetArticularPosition(VirtContext VC, float* pos);
```

DESCRIPTION

The `virtGetArticularPosition` function reads the articular position of the Virtuose, without any shift.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `pos` parameter corresponds to the position. It is an array of float, with length the number of axes of the Virtuose.

RETURN VALUE

In case of success, the `virtGetArticularPosition` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCOMPATIBLE_VERSION`
The embedded software version is lower than 3.60.

SEE ALSO

virtSetArticularPosition

virtGetArticularSpeed

virtSetArticularSpeed

virtSetArticularForce

NAME

virtSetArticularPosition – Send articular position order.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetArticularPosition(VirtContext VC, float* pos);
```

DESCRIPTION

The `virtSetArticularPosition` function sends an articular position order.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `pos` parameter corresponds to the position. It is an array of float, with length the number of axes of the Virtuose.

RETURN VALUE

In case of success, the `virtSetArticularPosition` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCOMPATIBLE_VERSION`
The embedded software version is lower than 3.60.

SEE ALSO

virtGetArticularPosition

virtGetArticularSpeed

virtSetArticularSpeed

virtSetArticularForce

NAME

virtGetArticularSpeed – Read articular speed.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetArticularSpeed(VirtContext VC, float* speed);
```

DESCRIPTION

The `virtGetArticularSpeed` function reads the articular speed of the Virtuose.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `speed` parameter corresponds to the articular speed. It is an array of float, with length the number of axes of the Virtuose.

RETURN VALUE

In case of success, the `virtGetArticularSpeed` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The parameter `VC` does not correspond to a valid `VirtContext` object.

`VIRT_E_INCOMPATIBLE_VERSION`
The embedded software version is lower than 3.60.

SEE ALSO

virtSetArticularSpeed

virtGetArticularPosition

virtSetArticularPosition

virtSetArticularForce

NAME

virtSetArticularSpeed – Send articular speed order.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetArticularSpeed(VirtContext VC, float* speed);
```

DESCRIPTION

The `virtSetArticularSpeed` function sends an articular speed order to the Virtuose.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `speed` parameter corresponds to the articular speed. It is an array of float, with length the number of axes of the Virtuose.

RETURN VALUE

In case of success, the `virtSetArticularSpeed` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCOMPATIBLE_VERSION`
The embedded software version is lower than 3.60.

SEE ALSO

virtGetArticularSpeed

virtGetArticularPosition

virtSetArticularPosition

virtSetArticularForce

NAME

virtSetArticularForce – Send articular force order.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetArticularForce(VirtContext VC, float* force);
```

DESCRIPTION

The `virtSetArticularForce` function sends an articular force order to the Virtuose.
The type of command must be `COMMAND_TYPE_ARTICULAR_IMPEDANCE`.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `force` parameter corresponds to the articular force. It is an array of float, with length the number of axes of the Virtuose.

RETURN VALUE

In case of success, the `virtSetArticularForce` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCOMPATIBLE_VERSION`
The embedded software version is lower than 3.60.

SEE ALSO

virtGetArticularPosition

virtSetArticularPosition

virtGetArticularSpeed

virtSetArticularSpeed

NAME

virtEnableForceFeedback – Activate or deactivate force-feedback.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtEnableForceFeedback(VirtContext VC, int enable);
```

DESCRIPTION

The `virtEnableForceFeedback` function activates or deactivates the force-feedback on the device. By default, the force feedback is authorized. This function allows to deactivate the force-feedback temporarily, i.e. the device is under power but transparent. Do not activate the force-feedback if the manipulated object is in collision.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `enable` parameter corresponds to the force-feedback authorization. The value 1 means the authorization is granted, the value 0 means no authorization.

RETURN VALUE

In case of success, the `virtEnableForceFeedback` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

NAME

virtSaturateTorque – Saturate cartesian efforts (force and torque).

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSaturateTorque(VirtContext VC,
                       float forceThreshold,
                       float momentThreshold);
```

DESCRIPTION

The `virtSaturateTorque` function saturates the Cartesian effort, in translation and rotation, applied to the device.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `forceThreshold` parameter corresponds to the Cartesian effort in translation. It should not be higher than the initial effort defined by HAPTION for each device. For instance, 35 N is the initial value for a Virtuose 6D.
The `momentThreshold` parameter corresponds to the Cartesian effort in rotation. It should not be higher than the initial effort defined by HAPTION for each device. For instance, 3.3 Nm is the initial value for a Virtuose 6D.

RETURN VALUE

In case of success, the `virtSaturateTorque` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

NAME

virtSetFingerTipSpeed – Project specific

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetFingerTipSpeed(VirtContext VC, float *speed);
```

DESCRIPTION

The `virtSetFingerTipSpeed` function was define for the specific. It should not be used for any standard device.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `speed` parameter is a table of 5 floating point numbers.

RETURN VALUE

In case of success, the `virtSetPwmOutput` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The parameter `VC` does not correspond to a valid `VirtContext` object.

SEE ALSO

NAME

virtGetSimulationStiffness – Get the current simulation stiffness of the simulation

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetSimulationStiffness(VirtContext VC,
                               float *stiffness);
```

DESCRIPTION

The `virtGetSimulationStiffness` function returns the stiffness gain currently applied by the simulation.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `stiffness` parameter is filled by the function. It corresponds to the stiffness gain of the simulation in N.

RETURN VALUE

In case of success, the `virtGetSimulationStiffness` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtGetSimulationDamping

NAME

virtGetSimulationDamping – Get the current simulation damping of the simulation

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetSimulationDamping(VirtContext VC, float *damping);
```

DESCRIPTION

The `virtGetSimulationDamping` function returns the damping gain currently applied by the simulation.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `damping` parameter is filled by the function. It corresponds to the damping gain of the simulation in Nm.

RETURN VALUE

In case of success, the `virtGetSimulationDamping` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtGetSimulationStiffness

5.7 Hybrid position/speed control

NAME

virtActiveSpeedControl – Activate hybrid force/speed control in translation.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtActiveSpeedControl (VirtContext VC,
                            float radius,
                            float speedFactor);
```

DESCRIPTION

The `virtActiveSpeedControl` function activates a hybrid control mode, with a standard position/force control inside a sphere, and a simple speed control (similar to joystick control) outside the sphere.
The sphere is defined by its centre, which is always identical to the centre of the device workspace, and its radius, which can be specified by the user.
Inside the sphere, the control mode is unchanged.
Outside the sphere, the positions given by `virtGetPosition` and `virtGetAvatarPosition` vary proportionally to the distance to the sphere affected by the parameter `speedFactor`, in the same directions, while a constant force tends to bring back the end-effector of the Virtuose inside the sphere.
The typical values of the parameter `radius` in meters are 0.1 for a device of type Virtuose 3D15-25, and 0.2 for a Virtuose 6D35-45.
The default value for `speedFactor` is 1.0.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `radius` parameter corresponds to the radius of the sphere, given in meters.
The `speedFactor` parameter corresponds to a scaling of the speed.

RETURN VALUE

In case of success, the `virtActiveSpeedControl` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCORRECT_VALUE`
One of the parameters is negative or null.

SEE ALSO

virtDeactiveSpeedControl

NAME

virtDeactiveSpeedControl – Deactivate hybrid position/speed control in translation.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtDeactiveSpeedControl (VirtContext VC);
```

DESCRIPTION

The `virtDeactiveSpeedControl` function leaves the hybrid position/speed control mode, previously activated by `virtActiveSpeedControl`.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

RETURN VALUE

In case of success, the `virtDeactiveSpeedControl` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtActiveSpeedControl

NAME

virtActiveRotationSpeedControl – Activate rotation speed control.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtActiveRotationSpeedControl(VirtContext VC,
                                   float angle,
                                   float speedFactor);
```

DESCRIPTION

The `virtActiveRotationSpeedControl` function activates a hybrid control mode, with a standard position/force control inside a cone, and a simple speed control outside the cone.
Outside the cone, the positions given by `virtGetPosition` and `virtGetAvatarPosition` vary proportionally to the distance to the cone affected by the parameter `speedFactor`, in the same directions, while a constant torque tends to bring back the end-effector of the Virtuose inside the cone.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `angle` parameter corresponds to the angle of the cone.
The `speedFactor` parameter corresponds to a scaling of the rotation speed.

RETURN VALUE

In case of success, the `virtActiveRotationSpeedControl` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCOMPATIBLE_VERSION`
The embedded software version is lower than 3.55.

`VIRT_E_INCORRECT_VALUE`
One of the parameters is negative.

SEE ALSO

virtDeactiveRotationSpeedControl

NAME

virtDeactiveRotationSpeedControl – Deactivate rotation speed control.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtDeactiveRotationSpeedControl(VirtContext VC);
```

DESCRIPTION

The `virtDeactiveRotationSpeedControl` function leaves the hybrid position/speed control mode, previously activated by `virtActiveRotationSpeedControl`.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

RETURN VALUE

In case of success, the `virtDeactiveRotationSpeedControl` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCOMPATIBLE_VERSION`
The embedded software version is lower than 3.20.

SEE ALSO

virtActiveRotationSpeedControl

NAME

virtIsInSpeedControl – Test if the operator is in speed control mode.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtIsInSpeedControl(VirtContext VC,
                         int* translation, int* rotation);
```

DESCRIPTION

The `virtIsInSpeedControl` function test if the operator controls a virtual object in position or in speed mode, in translation and/or in rotation.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `translation` parameter corresponds to the translation. A value of 0 indicates the position mode (position of the end-effector inside of the sphere). A value of 1 indicates the speed mode (position of the end-effector outside of the sphere).
The `rotation` parameter corresponds to the rotation. A value of 0 indicates the position mode (orientation of the end-effector inside the cone). A value of 1 indicates the speed mode (orientation of the end-effector outside the cone).

RETURN VALUE

In case of success, the `virtIsInSpeedControl` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCOMPATIBLE_VERSION`
The embedded software version is lower than 3.20.

SEE ALSO

NAME

virtSetForceInSpeedControl – Set the force in speed control.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetForceInSpeedControl(VirtContext VC,
                               float force);
```

DESCRIPTION

The `virtSetForceInSpeedControl` function set the force in translation when the operator controls a virtual object in speed control.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `force` parameter corresponds to the force in translation. This force is not saturate, so it is recommended that the force be smaller than the nominal force of the device.

RETURN VALUE

In case of success, the `virtSetForceInSpeedControl` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCOMPATIBLE_VERSION`
The embedded software version is lower than 3.60.

SEE ALSO

virtSetTorqueInSpeedControl

NAME

virtSetTorqueInSpeedControl – Set the force in speed control.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetTorqueInSpeedControl(VirtContext VC,
                                float torque);
```

DESCRIPTION

The `virtSetTorqueInSpeedControl` function set the torque (in rotation) when the operator controls a virtual object in speed control.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `torque` parameter corresponds to the torque in rotation. This torque is not saturate, so it is recommended the force to be smaller than the nominal torque of the device.

RETURN VALUE

In case of success, the `virtSetTorqueInSpeedControl` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCOMPATIBLE_VERSION`
The embedded software version is lower than 3.60.

SEE ALSO

virtSetForceInSpeedControl

NAME

virtGetCenterSphere – Read the centre position of the sphere.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetCentreSphere(VirtContext VC, float* pos);
```

DESCRIPTION

The `virtGetCentreSphere` function reads the centre position of the sphere in case of speed control. This centre moves when the device is in shift, otherwise it stays still.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `pos` parameter corresponds to the centre position of the sphere. It is represented as an array of 7 floating-points.

RETURN VALUE

In case of success, the `virtGetCentreSphere` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtGetAxisOfRotation

NAME

virtGetAxisOfRotation – Read the rotation axis of the pinch.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtGetAxisOfRotation(VirtContext VC, float* axe);
```

DESCRIPTION

The `virtGetAxisOfRotation` function reads the axis of rotation of the pinch in the environment frame. In case of speed control on the rotations, this axis shows when the operation gets out of the cone and switch in speed mode. The rotation vector norm in the cone limit corresponds to the radius of the sphere defined in parameters of the speed mode activation function.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `axe` parameter corresponds to the rotation vector. It is represented as an array of 3 floating-points.

RETURN VALUE

In case of success, the `virtGetAxisOfRotation` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtActiveSpeedControl

virtActiveRotationSpeedControl

virtGetCenterSphere

virtDeactiveSpeedControl

virtDeactiveRotationSpeedControl

5.8 Virtual mechanism management

NAME

virtVmSetType– Select one type of virtual mechanism.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtVmSetType(VirtContext VC, virtVmType type);
```

DESCRIPTION

The `virtVmSetType` function selects the type of virtual mechanism to be used. This function is only available in `COMMAND_TYPE_VIRTMECH` control mode.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `type` parameter corresponds to the type of virtual mechanism. The available types are:

`VM_TYPE_CartMotion`: Virtual guide with 1 degree of freedom, which corresponds to a line segment (in 6 degrees-of-freedom), joining the current position of the end-effector to the centre of the virtual mechanism base frame.

`VM_TYPE_Spline`: Virtual guide with 1 degree of freedom, based on a 6D spline curve (created with `virtTrajRecordStart` and `virtTrajRecordStop`)

`VM_TYPE_Rx`: Virtual guide with 1 degree of freedom, i.e. one rotation around the X axis of the virtual mechanism base frame.

`VM_TYPE_Tx`: Virtual guide with 1 degree of freedom, i.e. one translation along the X axis of the virtual mechanism base frame.

`VM_TYPE_TxTy`: Virtual guide with 2 degrees of freedom, corresponding to two translations along axes X and Y of the virtual mechanism base frame.

`VM_TYPE_TxRx`: Virtual guide with 2 degrees of freedom, corresponding to one translation and one rotation around axis X of the virtual mechanism base frame.

`VM_TYPE_RxRyRz`: Virtual guide with 3 degrees of freedom, corresponding to three rotations around the centre of the virtual mechanism base frame.

`VM_TYPE_Crank`: Virtual guide with 2 degrees of freedom, corresponding to the movement of a crank around axis X of the virtual mechanism base frame.

RETURN VALUE

In case of success, the function `virtVmSetType` returns `0`. Otherwise, it returns `-1` and the function `virtGetErrorCode` gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The parameter `VC` does not correspond to a valid `VirtContext` object

`VIRT_E_INCORRECT_VALUE`
The value of parameter `type is` incorrect.

SEE ALSO

virtVmSetBaseFrame

NAME

virtVmActivate – Activate the selected type of virtual mechanism.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtVmActivate(VirtContext VC);
```

DESCRIPTION

The `virtVmActivate` function activates the virtual mechanism selected before with `virtVmSetType`.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

RETURN VALUE

In case of success, the `virtVmActivate` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

SEE ALSO

virtVmDeactivate

NAME

virtVmDeactivate – Deactivate the current virtual mechanism.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtVmDeactivate(VirtContext VC);
```

DESCRIPTION

The `virtVmDeactivate` function deactivates the virtual mechanism previously activated.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

RETURN VALUE

In case of success, the `virtVmDeactivate` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

SEE ALSO

virtVmActivate

NAME

virtTrajSetSamplingTimeStep – Set the value of the trajectory sampling period.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtTrajSetSamplingTimeStep(VirtContext VC,
                                float timeStep,
                                unsigned int* recordTime) ;
```

DESCRIPTION

The `virtTrajSetSamplingTimeStep` function sets the value of the trajectory sampling period and maximum recording time, to be used in conjunction of `virtTrajRecordStart` and `virtTrajRecordStop` functions.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `timeStep` parameter corresponds to the sampling period expressed in seconds.
The `recordTime` parameter corresponds to the maximum recording time expressed by seconds.

RETURN VALUE

In case of success, the `virtTrajSetSamplingTimeStep` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

`VIRT_E_INCORRECT_VALUE`
The value of at least one parameter is incorrect. This error is generated by these following cases:

- Value is negative or null
- Value is smaller than the timestep of the Virtuose controller

SEE ALSO

virtTrajRecordStart

virtTrajRecordStop

NAME

virtTrajRecordStart – Start recording the trajectory.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtTrajRecordStart (VirtContext VC);
```

DESCRIPTION

The `virtTrajRecordStart` function starts the recording of the trajectory.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

RETURN VALUE

In case of success, the `virtTrajRecordStart` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

SEE ALSO

virtTrajRecordStop

virtTrajSetSamplingTimeStep

NAME

virtTrajRecordStop – Stop recording the trajectory.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtTrajRecordStop (VirtContext VC);
```

DESCRIPTION

The `virtTrajRecordStop` function stops the recording of the trajectory started by `virtTrajRecordStart`.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

RETURN VALUE

In case of success, the `virtTrajRecordStop` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

SEE ALSO

virtTrajRecordStart

virtTrajSetSamplingTimeStep

NAME

virtVmStartTrajSampling – Initialize the transfer of the recorded trajectory.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtVmStartTrajSampling (VirtContext VC,
                             unsigned int nbSamples);
```

DESCRIPTION

The `virtVmStartTrajSampling` function starts the transfer of the control points of a trajectory previously recorded.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `nbSamples` parameter corresponds to the number of points requested. In case the number of points is lower than the total number of recorded points, a new sampling is done in order to fit to the requested number of control points.

RETURN VALUE

In case of success, the `VirtVmStartTrajSampling` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCORRECT_VALUE`
The `nbSamples` parameter is negative or higher than 3000 points.

SEE ALSO

virtVmGetTrajSamples

NAME

virtVmGetTrajSamples – Transfer the trajectory control points.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtVmGetTrajSamples (VirtContext VC, float* samples);
```

DESCRIPTION

The `virtVmGetTrajSamples` function transfers the control points of a previously recorder trajectory. This function blocks until the end of the transfer of the control points from the Virtuose controller

PARAMETERS

The parameter `VC` corresponds to a `VirtContext` object returned by the `virtOpen` function.

The `samples` parameter corresponds to a pre-allocated array of floats of size `7*nbSamples`, where `nbSamples` is the number of control points specified with the `virtVmStartTrajSampling` function, each control point being defined as a displacement vector of 7 components.

RETURN VALUE

In case of success, the `virtVmGetTrajSamples` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

SEE ALSO

virtVmStartTrajSampling

NAME

virtVmSetDefaultToCartesianPosition – Set the default behaviour to blocked mode.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtVmSetDefaultToCartesienPosition (VirtContext VC);
```

DESCRIPTION

The `virtVmSetDefaultToCartesienPosition` function sets the default behaviour of the haptic device to blocked mode. When no virtual guide is active, the haptic device cannot be moved. This function is only available in `COMMAND_TYPE_VIRTMECH` control mode.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

RETURN VALUE

In case of success, the `VirtVmSetDefaultToCartesianPosition` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

SEE ALSO

virtSetDefaultToTransparentMode

virtVmActivate

virtVmDeactivate

NAME

virtVmSetDefaultToTransparentMode – Set the default behaviour to free motion.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtVmSetDefaultToTransparentMode (VirtContext VC);
```

DESCRIPTION

The `virtSetVmDefaultToTransparentMode` function sets the default behaviour of the haptic device to free motion. When no virtual guide is active, the haptic device can be moved freely. This function is only available in `COMMAND_TYPE_VIRTMECH` control mode.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

RETURN VALUE

In case of success, the `virtVmSetDefaultToTransparentMode` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object

SEE ALSO

virtSetDefaultToCartesianMode

virtVmActivate

virtVmDeactivate

NAME

virtVmSetBaseFrame – Modify the position of the virtual mechanism base frame.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtVmSetBaseFrame (VirtContext VC, float *base);
```

DESCRIPTION

The `virtVmSetBaseFrame` function changes the position of the virtual mechanism base frame.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `base` parameter corresponds to the transformation from the environment reference frame to the virtual mechanism base frame, expressed as a displacement vector of 7 components (see glossary).

RETURN VALUE

In case of success, the `VirtVmSetBaseFrame` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtVmType

virtVmGetBaseFrame

virtVmSetBaseFrameToCurrentFrame

NAME

virtVmGetBaseFrame – Read the current position of the virtual mechanism base frame.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtVmGetBaseFrame (VirtContext VC, float *base);
```

DESCRIPTION

The `VirtVmGetBaseFrame` function returns the current position of the virtual mechanism base frame.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `base` parameter corresponds to the reference frame of the base. It is expressed compared to the reference frame of environment.

RETURN VALUE

In case of success, the `VirtVmGetBaseFrame` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtVmSetBaseFrame

NAME

virtVmWaitUpperBound – Wait until the upper bound of a virtual guide has been reached.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtVmWaitUpperBound (VirtContext VC);
```

DESCRIPTION

The `VirtVmWaitUpperBound` function is available only in control mode `COMMAND_TYPE_VIRTMECH` and for virtual guides of type `VM_TYPE_CartMotion` and `VM_TYPE_Spline`.
When a virtual guide is active, the function blocks until the upper bound of the virtual guide is reached, or the virtual guide is deactivated.
A typical use of this function is to execute trajectories defined as sequences of virtual guides.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

RETURN VALUE

In case of success, the `virtVmWaitUpperBound` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCORRECT_VM_TYPE`
The type of virtual guide is incorrect.

`VIRT_E_WRONG_MODE`
The type of control mode is incorrect.

`VIRT_E_CALL_TIME`
The function has been called before the activation of the virtual guide.

SEE ALSO

virtVmSetType

virtVmActivate

virtVmDeactivate

NAME

virtVmSetRobotMode – Activate or deactivate the robot mode.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtVmSetRobotMode(VirtContext VC, int OnOff);
```

DESCRIPTION

The `virtVmSetRobotMode` function activates or deactivates the automatic displacement of the device for the Segments and Splines virtual mechanisms.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

The `OnOff` parameter corresponds to the activation or deactivation of the robot mode. The value 1 is for activation and 0 for deactivation. By default, the robot mode is deactivated.

RETURN VALUE

In case of success, the `virtVmSetRobotMode` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_WRONG_MODE`
The type of control mode is not `COMMAND_TYPE_VIRTMECH`.

SEE ALSO

NAME

virtVmSaveCurrentSpline – Save the current Spline trajectory.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtVmSaveCurrentSpline(VirtContext VC, char * name);
```

DESCRIPTION

The `virtVmSaveCurrentSpline` function saves the spline trajectory in a text file.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

The `name` parameter corresponds to the name of the created file.

RETURN VALUE

In case of success, the `virtVmSaveCurrentSpline` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_WRONG_MODE`
The type of control mode is not `COMMAND_TYPE_VIRTMECH`.

`VIRT_E_INCORRECT_VM_TYPE`
The type of virtual mechanism is not `VM_TYPE_Spline`.

SEE ALSO

virtVmLoadSpline

virtVmDeleteSpline

NAME

virtVmLoadSpline – Load a spline trajectory.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtVmLoadSpline(VirtContext VC, char * file_name);
```

DESCRIPTION

The `virtVmLoadSpline` function loads a saved spline trajectory.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

The `file_name` parameter corresponds to the name of the file to load.

RETURN VALUE

In case of success, the `virtVmLoadSpline` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_WRONG_MODE`
The type of control mode is not `COMMAND_TYPE_VIRTMECH`.

`VIRT_E_INCORRECT_VM_TYPE`
The type of virtual mechanism is not `VM_TYPE_Spline`.

SEE ALSO

virtVmSaveCurrentSpline

virtVmDeleteSpline

NAME

virtVmDeleteSpline – Delete a spline trajectory.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtVmDeleteSpline(VirtContext VC, char * file_name);
```

DESCRIPTION

The `virtVmDeleteSpline` function deletes a saved spline trajectory.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

The `file_name` parameter corresponds to the name of the file to delete.

RETURN VALUE

In case of success, the `virtVmDeleteSpline` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_WRONG_MODE`
The type of control mode is not `COMMAND_TYPE_VIRTMECH`.

`VIRT_E_INCORRECT_VM_TYPE`
The type of virtual mechanism is not `VM_TYPE_Spline`.

SEE ALSO

virtVmSaveCurrentSpline

virtVmLoadSpline

NAME

virtVmSetBaseFrameToCurrentFrame – Set the base frame of the virtual mechanism.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtVmSetBaseFrameToCurrentFrame(VirtContext VC);
```

DESCRIPTION

The `virtVmSetBaseFrameToCurrentFrame` function set the virtual mechanism base frame to the avatar frame; both frames are expressed in the same environments. This function is only accessible in `COMMAND_TYPE_VIRTMECH` control mode.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.

RETURN VALUE

In case of success, the `virtVmSetBaseFrameToCurrentFrame` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_WRONG_MODE`
The type of control mode is not `COMMAND_TYPE_VIRTMECH`.

SEE ALSO

virtVmSetBaseFrame

5.9 Haptic texture

NAME

virtSetTexture – Simulate a haptic texture.

SYNOPSIS

```
#include "virtuoseAPI.h"
#include "virtuoseAPI.h"
int virtSetTexture (VirtContext VC,
                    float *H_texture_p_env,
                    float* intensite,
                    int reinit);
```

DESCRIPTION

The `virtSetTexture` function simulates a haptic texture, related to a gradient in a greyscale value. A haptic texture gives an impression of surface roughness, without actually modifying the geometry. Light colours are rendered as bumps and dark as holes.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `H_texture_p_env` parameter corresponds to the position of the contact point with the surface, expressed as a displacement vector of 7 components in the coordinates of the environment reference frame.
The `intensity` parameter corresponds to the greyscale value at the contact point.
The `reinit` parameter cancels the current texture rendering. It should be set to 1 once when entering contact, and then to 0 for texture rendering.

RETURN VALUE

In case of success, the `virtSetTexture` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

`VIRT_E_INCORRECT_VALUE`
The `intensity` parameter is not a value between 0 and 1.

SEE ALSO

virtConvertRGBToGrayscale

virtSetTextureForce

NAME

virtSetTextureForce – Apply a force simulating a haptic texture.

SYNOPSIS

```
#include "virtuoseAPI.h"
int virtSetTextureForce (VirtContext VC,
                         float *W_texture_p_env_r_ps);
```

DESCRIPTION

The `virtSetTextureForce` function adds a force directly at the end-effector of the haptic device. It is commonly used for rendering haptic textures or force fields, which cannot be rendered by the `virtSetTexture` function.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `W_texture_p_env_r_ps` parameter corresponds to the force to be added, expressed as a dynamic tensor of 6 components with respect to the Virtuose end-effector, in the coordinates of the environment reference frame.

RETURN VALUE

In case of success, the v`irtSetTextureForce` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtSetTexture

NAME

virtConvertRGBToGrayscale – Convert a RGB colour to greyscale.

SYNOPSIS

```
#include "virtuoseAPI.h"
#include "virtuoseAPI.h"
int virtConvertRGBToGrayscale (VirtContext VC,
                               float *rgb,
                               float* gray);
```

DESCRIPTION

The `virtConvertRGBToGrayscale` function converts a colour defined as RGB (red, green, blue) values to a greyscale colour. The following ratios are used:

- 29.9% de rouge,
- 58.7% de vert,
- 11.4% de bleu.

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `rgb` parameter is a table of three components (varying between 0 and 1), of which the first corresponds to the red colour, the second with the green colour and the third with the blue colour.
The `gray` parameter is a single floating point value, set by the function.

RETURN VALUE

In case of success, the `virtConvertRGBToGrayscale` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtVmSetTexture

5.10 Utility

NAME

virtConvertDisplToTransformationMatrix – Convert a displacement vector into a matrix transform.

SYNOPSIS

```
#include "virtuoseAPI.h"

#define virtConvertDisplToTransformationMatrix
  virtConvertDeplToHomogeneMatrix

int virtConvertDeplToHomogeneMatrix (VirtContext VC,
                                     float* d,
                                     float* m);
```

DESCRIPTION

The `virtConvertDisplToTransformationMatrix` function converts a displacement vector given as 7 components (see glossary) into a 4x4 matrix transform stored in columns:

- | a0 a4 a8 a12 |
- | a1 a5 a9 a13 |
- | a2 a6 a10 a14 |
- | a3 a7 a11 a15 |

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `d` parameter is the displacement vector to convert.
The `m` parameter is the matrix transform, to be filled by the function.

RETURN VALUE

In case of success, the `virtConvertDisplToTransformationMatrix` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtConvertTransformationMatrixToDispl

NAME

virtConvertTransformationMatrixToDispl – Convert a matrix transform into a displacement vector.

SYNOPSIS

```
#include "virtuoseAPI.h"

#define virtConvertTransformationMatrixToDispl
     virtConvertHomogeneMatrixToDepl

int virtConvertHomogeneMatrixToDepl (VirtContext VC,
                                     float* d,
                                     float* m);
```

DESCRIPTION

The `virtConvertTransformationMatrixToDispl` function converts a 4x4 matrix transform into a displacement vector (see glossary). The matrix transform should be stored in columns:

- | a0 a4 a8 a12 |
- | a1 a5 a9 a13 |
- | a2 a6 a10 a14 |
- | a3 a7 a11 a15 |

PARAMETERS

The `VC` parameter corresponds to a `VirtContext` object returned by the `virtOpen` function.
The `m` parameter is the matrix transform to convert.
The `d` parameter is the displacement vector to be filled by the function.

RETURN VALUE

In case of success, the `virtConvertTransformationMatrixToDispl` function returns `0`. Otherwise, it returns `-1` and the `virtGetErrorCode` function gives access to an error code.

ERRORS

`VIRT_E_INVALID_CONTEXT`
The `VC` parameter does not correspond to a valid `VirtContext` object.

SEE ALSO

virtConvertDisplToTransformationMatrix

5.11 Desciption of the field digital inputs

When we use the *virtGetDigitalinputs* function described page 76, the parameter `tor_in` is set by the function. This value is a field of bits corresponding to the digital inputs of the arm.

Virtuose 3D/6D with Cobra handle:

Force feedback switch	bit 0
Presence sensor	bit 1
Middle button (Clutch)	bit 2
Left button	bit 3
Right button	bit 5

Virtuose 6D with TREH handle:

Force feedback switch	bit 0
Presence sensor	bit 1
Clutch lever (Δ)	bit 2
Gripper locking (P)	bit 3
Master-Slave coupling (Next)	bit 4
Gripper rotation +	bit 5
Gripper rotation -	bit 6
Stop	bit 7
Master-Slave decoupling (Wait)	bit 8
Redundancy +	bit 9
Redundancy -	bit 10
Precision +	bit 11
Precision -	bit 12
Gripper force +	bit 13
Gripper force -	bit 14
Calibration	bit 15

Virtuose 3D/6D Desktop:

Force feedback switch	bit 0
Presence sensor	bit 1
Left button (on the pad)	bit 3
Middle button (on the pad)	bit 6
Right button (on the pad)	bit 7
Button on the handle (Clutch)	bit 5

6 GLOSSARY

QUATERNION

The following formula transforms a rotation of angle θ and vector (a, b, c) into a quaternion:

$$\begin{cases} qx = a \sin\dfrac{\theta}{2} \\ qy = b \sin\dfrac{\theta}{2} \\ qz = c \sin\dfrac{\theta}{2} \\ qw = \cos\dfrac{\theta}{2} \end{cases}$$

Afterwards, it is possible to normalize the quaternion by dividing each of its four components by its norm, computed as follows:

$$n = \sqrt{qx^2 + qy^2 + qz^2 + qw^2}$$

All the quaternions are represented with the scalar part in last position:

$$\text{Quaternion} = [\text{qx} \quad \text{qy} \quad \text{qz} \quad \text{qw}]$$

DISPLACEMENT

A displacement is an array of 7 values: 3 for the translation and 4 for the rotation:

$$\text{disp} = [\text{x} \quad \text{y} \quad \text{z} \quad \text{qx} \quad \text{qy} \quad \text{qz} \quad \text{qw}]$$

CINEMATIC TENSOR OR TWIST

Considering a solid (S) attached to a reference frame$(R_1) = \{O_1, x_1, y_1, z_1\}$, in motion with respect to a fixed reference frame $(R_0) = \{O, x_0, y_0, z_0\}$.

The speed of any point M of (S) with respect to (R0) is given by:

$$\vec{V}(M \in S/R_0) = \left(\frac{dOM}{dt}\right)_{R_0} = \left(\frac{dOO_1}{dt}\right)_{R_0} + x\left(\frac{dx_1}{dt}\right)_{R_0} + y\left(\frac{dy_1}{dt}\right)_{R_0} + z\left(\frac{dz_1}{dt}\right)_{R_0}$$

The label R_0means that, when computing the speed, the base vectors of (R_0) are considered constant. We note:

$$\left(\frac{dOO_1}{dt}\right)_{R_0} = v_x\overrightarrow{x_0} + v_y\overrightarrow{y_0} + v_z\overrightarrow{z_0}$$

The rotation vector of (S) with respect to (R_0) is the vector Ω with the following properties:

$$\begin{cases} \left(\frac{dx_1}{dt}\right)_{R_0} = \overrightarrow{\Omega} \wedge \overrightarrow{x_1} \\ \left(\frac{dy_1}{dt}\right)_{R_0} = \overrightarrow{\Omega} \wedge \overrightarrow{y_1} \\ \left(\frac{dz_1}{dt}\right)_{R_0} = \overrightarrow{\Omega} \wedge \overrightarrow{z_1} \end{cases}$$

The vector Ω does not depend of the choice of (R_1) attached to (S).

We note:

$$\vec{\Omega} = \omega_x \overrightarrow{x_1} + \omega_y \overrightarrow{y_1} + \omega_z \overrightarrow{z_1}$$

We come to the following formula:

$$\vec{V}(M \in S/R_0) = \vec{V}(O_1/R_0) + \overrightarrow{\Omega} \wedge O_1\vec{M}$$

The speed of (S) is the expression of a tensor called cinematic tensor or twist, which reduction components are:

$$(VS/R_0)_{O_1} = \begin{Bmatrix} V\_(O_1 \in S/R_0) \\ \Omega\,(S/R_0) \end{Bmatrix} = (v_x, v_y, v_z, \omega_x, \omega_y, \omega_z)$$

In this documentation, we call it “speed of S with respect to O_1 expressed in R_0”.